

CV Lab Mid-Exam Cheat Sheet

Oct 7, 2026 · @ibrahim

0. Exam brief (read this first)

The exam is open book, closed internet, 3 questions, Q1 is MCQs, and it covers Lab Manuals 1 to 6, including Lab 2 photometric topics that were never uploaded.

Sir Talha's announcement (GCR, Oct 4)

Item	Rule	What it means for you
Format	Open book, strictly closed internet	No Stack Overflow, no docs site. Everything you need must be printed or in your head.
Allowed materials	Printed cheat sheets for Python and OpenCV only	Print this doc. Function signatures (Section 9) matter more than prose.
Layout	3 questions; Q1 = MCQs	Q1 is theory and function behaviour. Q2 and Q3 are almost certainly coding tasks.
Syllabus	Lab Manual 1 through Lab Manual 6	Labs 1, 3, 4, 5, 6 manuals plus all task sheets.
Lab Manual 2	Not on GCR, still examinable	Photometric / point transforms: Log, Gamma, Piecewise, Histogram . Covered in full in Section 4.
Detailed breakdown	Promised "later tonight or tomorrow morning"	Not in the files you shared. If it arrives, check it against Section 9.

How to use this sheet in the exam

1. Ctrl+F the operation name (for example `addWeighted`, `inRange`, `watershed`, `HoughCircles`).

2. Copy the block, change the filename, run it. Every block starts with its own imports and a safe image load.
3. Every task block ends with a side-by-side Matplotlib plot and an `imwrite` to `output/`, because the Lab 5 rules demand both.
4. Write the "why" answers from the **Analysis** notes under each task. Instructors grade the explanation as heavily as the code.

Six habits that save marks

- Always check `if img is None:` after `cv2.imread`. A wrong path returns `None` silently.
- Convert BGR to RGB before `plt.imshow`, or reds and blues swap.
- Pass `cmap='gray'` for single-channel images in Matplotlib.
- Do maths in `float32`, then `np.clip(..., 0, 255).astype(np.uint8)` before display. `uint8` arithmetic wraps around ($250 + 10 = 4$).
- OpenCV sizes are `(width, height)`; NumPy shapes are `(height, width)`. This is the number-one source of bugs.
- Label every subplot title with its parameter values (for example `Canny 50/150`). Graders read titles first.

Errors in the manuals (likely MCQ traps)

Where	What the manual says	What is actually true
Lab 3, p.3 list	Translation listed under perspective transformation	Translation is affine . The manual's own diagram on p.4 shows this correctly.
Lab 3, 5.3 table	<code>b</code> = horizontal shear, <code>c</code> = vertical shear	Correct, but with <code>d</code> = vertical scale and <code>a</code> = horizontal scale. Rotation needs all four together.
Lab 1, 11.9 text	Threshold 100, maxval 200 "represents pure white"	200 is light grey, not white. Pure white is 255.
Lab 1, 11.1 description	Says <code>cv2.imshow</code> and <code>waitKey(0)</code> are used	The code actually uses <code>plt.imshow</code> . <code>cv2.imshow</code> does not work inline in Jupyter.

Where	What the manual says	What is actually true
Lab 1, 11.5 comment	"(5, 5) is the kernel size"	The code passes (31, 31). Kernel sides must be odd.
Lab 4, 4.2.1.2	BORDER_CONSTANT gives "valid" (no padding)	filter2D always returns the same size. True valid output = crop $k//2$ pixels from each side.
Lab 4, 4.2.1.3	"Zero padding" using BORDER_REFLECT	BORDER_REFLECT mirrors pixels. Zero padding is BORDER_CONSTANT.
Lab 4, 4.2.1.4	cv2.filter2D(..., strides=(2,2))	filter2D has no strides argument; this raises an error. Stride = slice the output <code>[::2, ::2]</code> .
Lab 4, 3.6 code	contrast = filter2D(image, ones)	That is a local sum . Real contrast = local standard deviation, $\sqrt{E[x^2] - E[x]^2}$.
Lab 4, 3.6 code	image ** 2 on uint8	Overflows. Cast to float32 first.
Lab 4, 3.4 code	Orientation from arctan2 binned over 0 to 360	arctan2 returns -180 to 180. Add 360 to negatives first (<code>% 360</code>).
Lab 4, 3.6 text	"Higher energy = less uniform texture"	In GLCM terms energy is the opposite (high = uniform). For the manual's local sum of squares, high just means bright. State which definition you use.
Lab 4 HED	Here HED = Histogram of Edge Directions	Not Holistically-Nested Edge Detection (a deep network). Do not mix them up in MCQs.
Lab 5, 2.3.4 code	image[markers == -1] = [255,0,0] called "red"	Image is BGR, so [255,0,0] is blue . Red is [0,0,255].
Lab 5, 2.3.3 code	Seed (10, 10) indexed as image[x, y]	NumPy indexing is [row, col] = [y, x]. The code works only because it is consistent.
Lab 1, 11.10	"positive = counter-clockwise"	Correct for getRotationMatrix2D as displayed on screen (y axis points down).

1. Environment & repository setup

Install packages before the exam (the internet is closed during it), work in one VS Code Jupyter notebook, and load every image through one safe helper with relative paths.

VS Code + Jupyter in 6 steps (do this tonight)

1. Install Python 3.10 to 3.12 and VS Code. In VS Code install the **Python** and **Jupyter** extensions (Microsoft).
2. Open your exam folder: File → Open Folder.
3. Create a virtual environment in the terminal (Ctrl+`):
 - a. Windows: `python -m venv .venv` then `.venv\Scripts\activate`
 - b. macOS/Linux: `python3 -m venv .venv` then `source .venv/bin/activate`
4. Install everything now: `pip install opencv-contrib-python numpy matplotlib pandas scikit-image PyWavelets ipykernel`
5. Ctrl+Shift+P → "Create: New Jupyter Notebook". Top right → Select Kernel → choose `.venv`.
6. Run the check cell below. If it prints versions and `SIFT ok`, you are ready.

`opencv-contrib-python` is a superset of `opencv-python`. Install only one of them; having both installed causes import clashes. `cv2.SIFT_create()` is in the main package since OpenCV 4.4.

```
# Cell 1: environment check (Lab 1 Part 1.2 "Import & Validate")
import sys, cv2, numpy as np, matplotlib, pandas as pd
import matplotlib.pyplot as plt
print("Python      :", sys.version.split()[0])
print("OpenCV      :", cv2.__version__)
print("NumPy       :", np.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Pandas       :", pd.__version__)
test = np.zeros((50, 50, 3), np.uint8)          # tiny black test image
assert cv2.cvtColor(test, cv2.COLOR_BGR2GRAY).shape == (50, 50)
cv2.SIFT_create(); print("SIFT ok")
```

The Jupyter magic from Lab 1 Part 1.1 is `%pip install opencv-python` and `%pip install matplotlib`, run inside a cell. `%pip` installs into the notebook's own kernel; plain `!pip` may install somewhere else.

Repository layout (Lab 2 submission rules)

Your root must follow this blueprint exactly; never dump all files into the main folder.

```

StudentName_Medical_Imaging_Portfolio/
├── Task_1_Chest_XRay/
│   ├── data/                # only the X-ray images you used
│   ├── output/              # saved enhanced results
│   ├── xray_enhancement.ipynb
│   └── README.md            # pipeline explanation
├── Task_2_Cardiac_Fusion/
│   ├── data/  output/  modal_fusion.ipynb  README.md
├── Task_3_Echo_Analysis/
│   ├── data/  output/  realtime_echo.ipynb  README.md
├── requirements.txt          # global dependencies
└── README.md                # overview + setup instructions

```

- **100 MB rule:** GitHub rejects large files. Never upload whole datasets (EchoNet, Kaggle); copy only the sample images or short clips your notebook uses into `data/`.
- **Relative paths only:** `cv2.imread('data/sample_xray.png')`, never `C:/Users/...`.
- **requirements.txt** pins versions so a grader can rebuild your environment:

```

opencv-python==4.8.0.76
numpy==1.26.0
matplotlib==3.8.0
pandas==2.1.1
scikit-image==0.22.0
PyWavelets==1.5.0

```

Generate it from your working environment with `pip freeze > requirements.txt`, then delete lines you do not use.

Safe loading and saving (paste once at the top of every notebook)

```
import os, cv2, numpy as np, matplotlib.pyplot as plt
from pathlib import Path

BASE = Path.cwd()                                # the notebook's own folder in VS Code
/ Jupyter
OUT = BASE / "output"
OUT.mkdir(exist_ok=True)                        # create output/ if missing

def load(path, flag=cv2.IMREAD_COLOR):
    """Read an image or stop with a clear message. flag: IMREAD_COLOR /
    IMREAD_GRAYSCALE / IMREAD_UNCHANGED."""
    p = Path(path)
    img = cv2.imread(str(p), flag)
    if img is None:                                # imread NEVER raises; it returns None
        raise FileNotFoundError(f"Could not read {p.resolve()} (exists={
p.exists()}")
    return img

def save(name, img):
    ok = cv2.imwrite(str(OUT / name), img)    # extension decides format: .png
    lossless, .jpg lossy
    print("saved" if ok else "FAILED", OUT / name)
```

- `imread` fails silently on a wrong path, a wrong extension, or (on Windows) non-English characters in the path. If the path has Urdu or other non-ASCII characters, read it with `cv2.imdecode(np.fromfile(path, np.uint8), cv2.IMREAD_COLOR)`.
- Check where Python thinks you are: `print(os.getcwd())`. VS Code runs a notebook from the notebook's own folder by default.
- `cv2.imwrite` expects BGR. If you converted to RGB for plotting, convert back before saving, or save the BGR original.

No image on hand? Make one

A synthetic image keeps you working if a download link is dead.

```

img = np.full((400, 600, 3), 255, np.uint8)                # white
canvas
cv2.rectangle(img, (50, 50), (250, 200), (0, 0, 255), -1)  # filled
red box (BGR)
cv2.circle(img, (420, 250), 90, (255, 0, 0), -1)          # filled
blue circle
cv2.putText(img, "TEST", (60, 330), cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 0, 0),
3)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
grad = np.tile(np.linspace(0, 255, 600, dtype=np.uint8), (400, 1)) # left-
dark to right-bright ramp

```

2. Universal utilities & exam curveballs

These helpers solve the problems examiners add to make a task harder: unequal sizes, odd ROIs, live parameter tuning, blob measurements, folders of images and video.

2.1 `show()` : plot any number of images side by side

Handles BGR colour, grayscale and float arrays, so every later block just calls `show(...)`.

```

import cv2, numpy as np, matplotlib.pyplot as plt

def show(images, titles=None, cols=None, size=4, cmap="gray"):
    """images: list of BGR / gray / float arrays. Colour images are assumed
    BGR (OpenCV default)."""
    n = len(images); cols = cols or n; rows = int(np.ceil(n / cols))
    plt.figure(figsize=(size * cols, size * rows))
    for i, im in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        if im.ndim == 3:
            plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB)) # BGR -> RGB
        for Matplotlib
            else:
                plt.imshow(im, cmap=cmap) # 1-channel:
needs a colormap
            if titles: plt.title(titles[i], fontsize=11)
            plt.axis("off")
    plt.tight_layout(); plt.show()

```

2.2 Size and channel safety (before `add`, `addWeighted`, `bitwise_*`, `hconcat`)

All of these need both inputs to have the **same height, width, channel count and dtype**, or OpenCV raises `Sizes of input arguments do not match`.

```
def to_bgr(img):
    """1-channel -> 3-channel; 4-channel (PNG with alpha) -> 3-channel."""
    if img.ndim == 2:
        return cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
    if img.shape[2] == 4:
        return cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)
    return img

def match(a, b, mode="resize"):
    """Return b made compatible with a. mode='resize' stretches, 'pad' keeps
    aspect ratio with black borders."""
    b = to_bgr(b) if a.ndim == 3 else (cv2.cvtColor(b, cv2.COLOR_BGR2GRAY) if
    b.ndim == 3 else b)
    h, w = a.shape[:2]
    if mode == "resize":
        b = cv2.resize(b, (w, h), interpolation=cv2.INTER_AREA)      # dsize
    is (WIDTH, HEIGHT)
    else: # letterbox: scale to fit, then pad
        s = min(w / b.shape[1], h / b.shape[0])
        b = cv2.resize(b, (int(b.shape[1] * s), int(b.shape[0] * s)))
        top = (h - b.shape[0]) // 2; left = (w - b.shape[1]) // 2
        b = cv2.copyMakeBorder(b, top, h - b.shape[0] - top, left, w -
        b.shape[1] - left,
                                cv2.BORDER_CONSTANT, value=0)
    return b.astype(a.dtype)

# usage: blended = cv2.addWeighted(img1, 0.6, match(img1, img2), 0.4, 0)
```

Interpolation choice: `INTER_AREA` for shrinking (least aliasing), `INTER_LINEAR` (default) or `INTER_CUBIC` for enlarging, `INTER_NEAREST` for masks and label images (keeps values exactly 0 or 255).

`cv2.add` vs `+`: OpenCV **saturates** ($250 + 10 \rightarrow 255$); NumPy uint8 **wraps** ($250 + 10 \rightarrow 4$). Always use `cv2.add` / `cv2.addWeighted` or work in float and clip.

2.3 ROI coordinate maths

All slicing is `img[y1:y2, x1:x2]` (rows first). A slice is a **view**: editing it edits the original. Use `.copy()` to keep them separate.

```

h, w = img.shape[:2]
cx, cy = w // 2, h // 2                                # centre (x, y)

# Fixed-size centre ROI (e.g. 300x300), clamped so it never leaves the image
def center_roi(img, rw, rh):
    h, w = img.shape[:2]
    x1 = max((w - rw) // 2, 0); y1 = max((h - rh) // 2, 0)
    return img[y1:y1 + rh, x1:x1 + rw], (x1, y1)

# Central N% of the image (N = 50 keeps the middle half in each direction)
N = 50
rw, rh = int(w * N / 100), int(h * N / 100)
roi_pct = img[(h - rh) // 2:(h + rh) // 2, (w - rw) // 2:(w + rw) // 2]

# Quadrants
top_left, top_right = img[:cy, :cx], img[:cy, cx:]
bot_left, bot_right = img[cy:, :cx], img[cy:, cx:]

# Bottom 20% / top 10% / left third
bottom20 = img[int(h * 0.8):, :]
top10     = img[:int(h * 0.1), :]
left3rd   = img[:, :w // 3]

# Circular / elliptical ROI = a mask, not a slice
mask = np.zeros((h, w), np.uint8)
cv2.circle(mask, (cx, cy), min(h, w) // 3, 255, -1)                #
# filled white circle
# cv2.ellipse(mask, (cx, cy), (w//3, h//4), 0, 0, 360, 255, -1)    #
# (axes), angle, startAngle, endAngle
circle_only = cv2.bitwise_and(img, img, mask=mask)                #
outside -> black

# Apply an effect ONLY inside a non-rectangular mask
blurred = cv2.GaussianBlur(img, (25, 25), 0)
result = np.where(mask[..., None] == 255, blurred, img)            #
[... , None] broadcasts over channels

```

2.4 Live tuning with trackbars

`cv2.imshow` windows work from a VS Code notebook on your own laptop (a separate window opens). They do **not** work in Colab. Press `q` or Esc to close and read the final values.

```

img = load("data/smarties.png")                # any image
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
win = "HSV tuner"
cv2.namedWindow(win)
nothing = lambda v: None
for name, init, mx in [("H_lo", 20, 179), ("S_lo", 100, 255), ("V_lo", 100,
255),
                        ("H_hi", 35, 179), ("S_hi", 255, 255), ("V_hi", 255,
255)]:
    cv2.createTrackbar(name, win, init, mx, nothing) # OpenCV hue range is
0-179

while True:
    g = lambda n: cv2.getTrackbarPos(n, win)
    lo = np.array([g("H_lo"), g("S_lo"), g("V_lo")]); hi =
np.array([g("H_hi"), g("S_hi"), g("V_hi")])
    mask = cv2.inRange(hsv, lo, hi)
    cv2.imshow(win, np.hstack([img, cv2.bitwise_and(img, img, mask=mask)]))
    if cv2.waitKey(30) & 0xFF in (ord("q"), 27): break
print("lower", lo, "upper", hi)
cv2.destroyAllWindows()

```

Canny version: swap the trackbars for `low` (0 to 500) and `high` (0 to 500), and the body for `cv2.Canny(gray, g("low"), g("high"))`. For an odd kernel size from a trackbar use `k = 2 * g("k") + 1`.

No popup windows allowed? Use a slider inside the notebook instead (`pip install ipywidgets` beforehand):

```

from ipywidgets import interact
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
@interact(low=(0, 255, 5), high=(0, 255, 5))
def tune(low=50, high=150):
    show([gray, cv2.Canny(gray, low, high)], ["gray", f"Canny {low}/{high}"])

```

2.5 Contours: area, perimeter, box, hull, centroid

Contours need a **binary** image (white objects on black). The centroid comes from image moments:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}},$$

$$M_{00} = \text{area}$$

```

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, bw = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
# objects white
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# OpenCV 4: 2 return values
out = img.copy(); kept = 0
for c in cnts:
    area = cv2.contourArea(c)
    if area < 200: continue # filter out specks
    (tune this)
    peri = cv2.arcLength(c, True) # True = closed curve
    x, y, bw_, bh = cv2.boundingRect(c) # upright box
    aspect = bw_ / float(bh)
    hull = cv2.convexHull(c)
    solidity = area / cv2.contourArea(hull) # 1.0 = perfectly convex
    circularity = 4 * np.pi * area / peri ** 2 # 1.0 = perfect circle
    M = cv2.moments(c)
    cx, cy = int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"])
    approx = cv2.approxPolyDP(c, 0.02 * peri, True) # 4 points -> rectangle-
like
    cv2.rectangle(out, (x, y), (x + bw_, y + bh), (0, 255, 0), 2)
    cv2.drawContours(out, [hull], -1, (255, 0, 0), 1)
    cv2.circle(out, (cx, cy), 4, (0, 0, 255), -1)
    kept += 1
    print(f"#{kept}: area={area:.0f} peri={peri:.0f} AR={aspect:.2f} solid=
{solidity:.2f} "
          f"circ={circularity:.2f} centroid=({cx},{cy}) corners=
{len(approx)}")
print("objects:", kept)
show([img, bw, out], ["original", "binary", f"{kept} objects"])

```

- Rotated box: `rect = cv2.minAreaRect(c); box = np.int32(cv2.boxPoints(rect))`, then `cv2.drawContours(out, [box], 0, (0,255,255), 2)`.
- Enclosing circle: `(x, y), r = cv2.minEnclosingCircle(c)`.
- `RETR_EXTERNAL` = outer outlines only; `RETR_TREE` = include holes (nested).

Bounding boxes around a colour mask ("draw boxes around every yellow object"): build `mask = cv2.inRange(hsv, lo, hi)`, clean it with `cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5,5), np.uint8))`, then run the loop above on `mask` instead of `bw`.

2.6 Text that stays readable (solid background)

```
def label(img, text, org=(10, 30), scale=0.8, fg=(255, 255, 255), bg=(0, 0, 0), th=2):
    (tw, tht), base = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, scale, th)
    x, y = org # org = bottom-left of the text
    cv2.rectangle(img, (x - 4, y - tht - 6), (x + tw + 4, y + base + 2), bg, -1)
    cv2.putText(img, text, (x, y), cv2.FONT_HERSHEY_SIMPLEX, scale, fg, th, cv2.LINE_AA)
    return img
```

2.7 Batch: process every image in a folder

```
from pathlib import Path
exts = {".png", ".jpg", ".jpeg", ".bmp", ".tif"}
for p in sorted(Path("data").iterdir()):
    if p.suffix.lower() not in exts: continue
    im = cv2.imread(str(p))
    if im is None: print("skip corrupt", p.name); continue # corrupted files return None
    edges = cv2.Canny(cv2.cvtColor(im, cv2.COLOR_BGR2GRAY), 50, 150)
    cv2.imwrite(str(Path("output") / f"{p.stem}_edges.png"), edges)
```

2.8 Video: robust read loop, FPS, writer

```
import time
cap = cv2.VideoCapture("data/echo.mp4") # 0 = webcam
if not cap.isOpened(): raise IOError("cannot open video")
fps_in = cap.get(cv2.CAP_PROP_FPS) or 30 # some files report 0
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)); h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
writer = cv2.VideoWriter("output/out.mp4", cv2.VideoWriter_fourcc(*"mp4v"), fps_in, (w * 2, h))
```

```

prev, bad = time.time(), 0
while True:
    ok, frame = cap.read()
    if not ok:                                # end of file OR corrupted
frame
        bad += 1
        if bad > 5: break                    # give up after 5 misses
in a row
        continue
    bad = 0
    proc = cv2.cvtColor(cv2.Canny(frame, 50, 150), cv2.COLOR_GRAY2BGR) #
writer needs 3 channels
    combo = np.hstack([frame, proc])          # same height + channels
required
    now = time.time(); fps = 1 / max(now - prev, 1e-6); prev = now
    label(combo, f"FPS {fps:.1f}")
    writer.write(combo)                       # frame size MUST equal
the writer's (w*2, h)
    cv2.imshow("video", combo)
    if cv2.waitKey(int(1000 / fps_in)) & 0xFF == ord("q"): break
cap.release(); writer.release(); cv2.destroyAllWindows()

```

If the saved file is empty or 0 KB, the frame size or channel count did not match the writer. Fallback codec: `VideoWriter_fourcc(*"XVID")` with an `.avi` file.

3. Lab 01: Fundamentals & basic operations

Lab 1 tests whether you can load, convert, crop, annotate, threshold, rotate, blend and equalize an image, plus plain Python classes and dictionaries.

3.1 Theory you need for MCQs

A digital image is a 2-D function $f(x, y)$ whose coordinates and amplitude are finite and discrete. Stored as a NumPy array of M rows (height) by N columns (width), origin $(0, 0)$ at the **top-left**.

$$f(x,y)=\begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

Term	One-line meaning
Pixel	Smallest unit (picture element); holds colour/intensity.
Resolution	Width × height in pixels, e.g. 1920×1080.
Grayscale	One channel, 0 (black) to 255 (white).
RGB / BGR	Three channels; OpenCV stores BGR , Matplotlib expects RGB.
Histogram	Count of pixels at each intensity.
Feature / descriptor	Distinctive part of an image / a compact vector describing it.
Segmentation	Splitting an image into meaningful regions.
Image registration	Aligning images of the same scene to one coordinate system.
Optical flow	Motion of pixels between consecutive video frames.
Homography	Transform relating two images of the same planar surface.
SIFT / SURF / HOG	Scale-invariant keypoints / faster SIFT-like / gradient-orientation descriptor.

Fundamental steps of DIP (11): image acquisition, image enhancement, image restoration, colour image processing, wavelets and multi-resolution processing, compression, morphological processing, segmentation, representation and description, object recognition, all coordinated by the knowledge base.

- Enhancement = subjective, make it look better (brightness, contrast). Restoration = objective, undo a known degradation (blur, noise).
- Segmentation is called the hardest step. Representation and description follow segmentation. Recognition assigns labels from descriptors.
- Five purposes of DIP: visualization, sharpening and restoration, retrieval, measurement of pattern, recognition.

Libraries table (MCQ bait): OpenCV = general CV, C++/Python, high abstraction. scikit-image = image processing, medium. Pillow = basic image I/O, low. TorchVision / Detectron2 = PyTorch (Detectron2 = object detection). Keras = high-level API now part of TensorFlow. OpenVINO = Intel inference optimisation. Caffe = C++ CNN framework.

3.2 Core operations (manual 11.1 to 11.12)

```
import cv2, numpy as np, matplotlib.pyplot as plt
img = cv2.imread("Sukuna.jpeg")           # BGR, uint8, shape (H,
W, 3)
if img is None: raise SystemExit("Error: Image not found.")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)   # for plt.imshow
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # 1 channel; or
imread(path, cv2.IMREAD_GRAYSCALE)

half = cv2.resize(rgb, None, fx=0.5, fy=0.5) # scale factors,
keeps aspect ratio
fixed = cv2.resize(rgb, (640, 480))         # exact (width,
height)
blur = cv2.GaussianBlur(rgb, (31, 31), 0)   # odd kernel; sigma 0
= auto from size
h, w = rgb.shape[:2]
left = rgb[:, :w // 2]                      # crop = slicing [y,
x]
texted = rgb.copy()
cv2.putText(texted, "Domain Expansion", (150, 150), cv2.FONT_HERSHEY_SIMPLEX,
1.5, (255, 0, 0), 2)
#          img      text          org=bottom-left  font
scale colour(RGB here) thickness
ret, th = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY) # > 100 -> 255
else 0; ret = 100
M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1)         # centre, angle
(+ = CCW), scale
rot = cv2.warpAffine(rgb, M, (w, h))                       # corners cut
off, filled black
eq = cv2.equalizeHist(gray)                                # 8-bit 1-
channel only

plt.figure(figsize=(15, 6))
for i, (im, t) in enumerate([(rgb, "Original"), (gray, "Gray"), (blur, "Blur
31"), (left, "Left half"),
                             (texted, "Text"), (th, "Binary 100"), (rot, "Rot
```

```

45"), (eq, "Equalized"))]:
    plt.subplot(2, 4, i + 1); plt.imshow(im, cmap="gray" if im.ndim == 2 else
None)
    plt.title(t); plt.axis("off")
plt.tight_layout(); plt.show()
print("Original", (w, h), "Resized", half.shape[1::-1])    # print as width
x height

```

Blending two images (11.11): both must be the same size first.

```

img1 = cv2.imread("Sukuna.jpeg"); img2 = cv2.imread("Gojo.jpeg")
img1 = cv2.resize(img1, (img2.shape[1], img2.shape[0]))    # match img2's
(width, height)
added = cv2.add(img1, img2)                                # saturating sum ->
brighter, clips at 255
mixed = cv2.addWeighted(img1, 0.6, img2, 0.4, 0)           # 0.6*img1 +
0.4*img2 + gamma(0)

```

$$\text{dst} = \alpha \cdot \text{src}_1 + \beta \cdot \text{src}_2 + \gamma$$

Pandas statistics (11.8): flatten (H, W, 3) to (H·W, 3), one row per pixel.

```

import pandas as pd
flat = img.reshape(-1, 3)                                # BGR order!
df = pd.DataFrame(flat, columns=["B", "G", "R"])
print(df.describe())                                     # count, mean, std,
min, 25%, 50%, 75%, max

```

3.3 Task 1: GroceryManager (OOP + error handling)

```

class GroceryManager:
    def __init__(self):
        self.items = {}                                # name ->
{"quantity": int, "price": float}

    def add_item(self, item, quantity, price):
        if quantity <= 0 or price < 0:
            raise ValueError("Quantity must be > 0 and price >= 0")
        if item in self.items:                          # adding again
            increases the quantity
            self.items[item]["quantity"] += quantity
            self.items[item]["price"] = price

```

```

        else:
            self.items[item] = {"quantity": quantity, "price": price}
            print(f"Added {quantity} x {item} @ {price}")

    def remove_item(self, item):
        try:
            del self.items[item]
            print(f"Removed {item}")
        except KeyError:
            print(f"Error: '{item}' is not in the list") # required error
handling

    def view_list(self):
        if not self.items:
            print("List is empty"); return
        print(f"{'Item':<12}{'Qty':>5}{'Price':>10}{'Subtotal':>10}")
        for name, d in self.items.items():
            print(f"{name:<12}{d['quantity']:>5}{d['price']:>10.2f}
{d['quantity'] * d['price']:>10.2f}")

    def calculate_total(self):
        return sum(d["quantity"] * d["price"] for d in self.items.values())

gm = GroceryManager()
gm.add_item("Milk", 2, 250); gm.add_item("Eggs", 12, 30);
gm.add_item("Bread", 1, 180)
gm.remove_item("Butter") # triggers the error message
gm.view_list(); print("Total:", gm.calculate_total())

```

3.4 Task 2: nested student records

```
students = {
    "23K-0001": {"Name": "Ali", "Major": "AI", "Grades": [85, 90, 78]},
    "23K-0002": {"Name": "Sara", "Major": "CS", "Grades": [92, 88, 95]},
    "23K-0003": {"Name": "Hamza", "Major": "AI", "Grades": [70, 75, 80]},
}

def top_student(records):
    best_id = max(records, key=lambda sid: sum(records[sid]["Grades"]) /
len(records[sid]["Grades"]))
    return records[best_id]["Name"]

def search_by_major(records, major):
    found = [r["Name"] for r in records.values() if r["Major"].lower() ==
major.lower()]
    print(f"{major}:", ", ".join(found) if found else "no students")

print("Highest average:", top_student(students))    # Sara
search_by_major(students, "AI")                    # Ali, Hamza
```

Empty-grade guard if asked: `sum(g) / len(g) if g else 0.`

3.5 Task 3: safe loading + formatted title

```
image_path = "Sukuna.jpeg"
image = cv2.imread(image_path)
if image is None:
    print("Error: Image not found.")
else:
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(8, 6)); plt.axis("off")
    plt.title("King of Curses - Ryomen Sukuna", fontsize=16, color="darkred",
pad=15)
    plt.imshow(image_rgb); plt.show()
```

3.6 Task 4: 800×800 canvas, 5 concentric circles, bounding box, centre

```

canvas = np.zeros((800, 800, 3), dtype=np.uint8)           # (height, width,
channels), black
cx, cy = canvas.shape[1] // 2, canvas.shape[0] // 2       # (400, 400)
colors = [(0, 0, 255), (255, 255, 255)]                  # alternate red /
white (BGR)
radii = [300, 240, 180, 120, 60]                          # draw LARGEST first
so smaller ones sit on top
for i, r in enumerate(radii):
    cv2.circle(canvas, (cx, cy), r, colors[i % 2], thickness=-1)  # -1 =
filled
R = radii[0]
cv2.rectangle(canvas, (cx - R, cy - R), (cx + R, cy + R), (0, 255, 0), 3) #
box around outer circle
cv2.circle(canvas, (cx, cy), 5, (255, 0, 0), -1)          # mark centre
print("Canvas centre (x, y):", (cx, cy))
plt.imshow(cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)); plt.title("Target
board"); plt.axis("off"); plt.show()

```

Variations: rings only (outlines) → `thickness=3` instead of `-1`. Any number of rings → `radii = np.linspace(300, 60, n).astype(int)`. Rainbow colours → `colors = [tuple(int(c) for c in cv2.applyColorMap(np.uint8([i * 255 // n])), cv2.COLORMAP_HSV)[0, 0]) for i in range(n)]`.

3.7 Task 5: heavy blur + 300×300 centre ROI from both images

```

image = cv2.imread("highres.jpg")
if image is None: raise SystemExit("Error: Image not found.")
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
blurred = cv2.GaussianBlur(rgb, (25, 25), 0)
h, w = rgb.shape[:2]
x1, y1 = w // 2 - 150, h // 2 - 150                        # top-left of the
300x300 centre box
roi_orig = rgb[y1:y1 + 300, x1:x1 + 300]
roi_blur = blurred[y1:y1 + 300, x1:x1 + 300]
fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(roi_orig); ax[0].set_title("Original ROI", fontsize=14,
color="navy"); ax[0].axis("off")
ax[1].imshow(roi_blur); ax[1].set_title("Blurred ROI (25x25)", fontsize=14,
color="darkred"); ax[1].axis("off")
plt.show()

```

```
Curveball "blur ONLY the centre, keep the rest sharp": out = rgb.copy();
out[y1:y1+300, x1:x1+300] = cv2.GaussianBlur(out[y1:y1+300, x1:x1+300],
(25,25), 0).
```

3.8 Task 6: semi-transparent caption bar on the bottom 20%

```
image = cv2.imread("Sukuna.jpeg")
h, w = image.shape[:2]
overlay = image.copy()
cv2.rectangle(overlay, (0, int(h * 0.8)), (w, h), (255, 0, 0), -1) # solid
blue bar (BGR)
alpha = 0.4 # 0 =
invisible, 1 = solid
blended = cv2.addWeighted(overlay, alpha, image, 1 - alpha, 0)
cv2.putText(blended, "King of Curses", (20, int(h * 0.92)),
cv2.FONT_HERSHEY_SIMPLEX, w / 600, (255, 255, 255), 2,
cv2.LINE_AA) # scale with width
plt.imshow(cv2.cvtColor(blended, cv2.COLOR_BGR2RGB)); plt.axis("off");
plt.show()
```

Centred text: (tw, th), _ = cv2.getTextSize(text, font, scale, 2), then x = (w - tw) // 2, y = int(h * 0.9) + th // 2.

3.9 Task 7: global vs adaptive threshold, then rotate 45° at 0.8

```
gray = cv2.imread("document.jpg", cv2.IMREAD_GRAYSCALE)
_, glob = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
adapt = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 11, 2)
# maxVal method type
blockSize(odd) C
h, w = adapt.shape
M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 0.8) # 0.8 shrink keeps
corners inside
rotated = cv2.warpAffine(adapt, M, (w, h), borderValue=255) # white fill
(optional, looks cleaner)
show([gray, glob, adapt, rotated], ["Gray", "Global 127", "Adaptive Gaussian
11/2", "Rotated 45 @0.8"])
```

Why 0.8? A square rotated 45° needs a $\sqrt{2} \approx 1.41\times$ larger canvas; scaling by 0.8 shrinks it enough that most corners stay visible. To keep the **whole** image at scale 1, enlarge the canvas (Section 5.6).

3.10 Task 8: bitwise masking (AND, NOT, OR, XOR)

```
a = cv2.resize(cv2.imread("Sukuna.jpeg"), (500, 500))
b = cv2.resize(cv2.imread("Gojo.jpeg"), (500, 500))
mask = np.zeros((500, 500), np.uint8) # 1-channel,
black
pts = np.array([[250, 60], [440, 200], [370, 440], [130, 440], [60, 200]],
np.int32) # pentagon
cv2.fillPoly(mask, [pts], 255) # or
cv2.circle(mask, (250,250), 180, 255, -1)
fg = cv2.bitwise_and(a, a, mask=mask) # shape cut from
image A
inv = cv2.bitwise_not(mask) # 255 - mask
bg = cv2.bitwise_and(b, b, mask=inv) # background
from image B
combined = cv2.bitwise_or(fg, bg) # same as
cv2.add(fg, bg) here
x = cv2.bitwise_xor(mask, cv2.circle(np.zeros_like(mask), (250, 250), 150,
255, -1)) # XOR demo
show([a, b, mask, fg, bg, combined, x], ["A", "B", "Mask", "A AND mask", "B
AND NOT mask", "OR", "XOR"], cols=4)
```

Op	Rule per bit	Use
AND	1 only if both 1	Keep pixels inside a mask
OR	1 if either 1	Merge two non-overlapping layers
NOT	flips 0 ↔ 255	Invert a mask
XOR	1 if exactly one is 1	Show where two masks differ

The `mask` argument must be uint8, single channel, same height and width as the image.

3.11 Task 9: per-channel statistics with Pandas

```
import pandas as pd
rgb = cv2.cvtColor(cv2.imread("Sukuna.jpeg"), cv2.COLOR_BGR2RGB)
R, G, B = rgb[:, :, 0].flatten(), rgb[:, :, 1].flatten(), rgb[:, :, 2].flatten() # 2-D -> 1-D
df = pd.DataFrame({"Red": R, "Green": G, "Blue": B})
summary = df.describe()
print(summary)
print(summary.loc[["mean", "std", "min", "max"]]) # just the four asked for
print("Brightest channel on average:", df.mean().idxmax())
```

4. Lab 02: Photometric (point) transforms & medical imaging

A point transform changes each pixel's value using only that pixel's value, $s = T(r)$, never its neighbours; Sir Talha named Log, Gamma, Piecewise and Histogram as the examinable four.

4.1 The big picture

- r = input intensity, s = output intensity, both in $[0, L-1]$ with $L = 256$ for 8-bit images.
- **Geometric** transforms move pixels (where). **Photometric** transforms change values (what). Point transforms are photometric.
- A point transform is just a lookup table (LUT) of 256 entries, so `cv2.LUT(img, table)` applies any of them fast.

Transform	Formula	Brightens	Typical use
Negative	$s = L - 1 - r$	Inverts	Mammograms, white detail in dark areas
Log	$s = c \cdot \log(1 + r)$	Dark regions strongly	Huge dynamic range (Fourier spectra), dark X-rays
Inverse log	$s = c \cdot (e^r - 1)$	Bright regions	Opposite of log
Gamma $\gamma < 1$	$s = c \cdot r^\gamma$	Darks and midtones	Underexposed images, soften bone glare
Gamma $\gamma > 1$	$s = c \cdot r^\gamma$	Darkens	Washed-out images

Transform	Formula	Brightens	Typical use
Piecewise linear	segments between $(r_1, s_1), (r_2, s_2)$	You choose	Contrast stretching, thresholding, slicing
Histogram equalization	$s = (L-1) \cdot \text{CDF}(r)$	Spreads clumps	Low-contrast images, automatic

4.2 Log transformation

$$s = c \cdot \log(1 + r), \quad \text{where } c = \frac{255}{\log(1 + r_{\max})}$$

The `+1` avoids $\log(0)$. The constant c rescales so the brightest input maps to 255. Log stretches dark values (0 to 50 spreads over roughly 0 to 180) and compresses bright ones.

```
import cv2, numpy as np
gray = cv2.imread("xray.png", cv2.IMREAD_GRAYSCALE)
f = gray.astype(np.float32) # float: avoid uint8 overflow
c = 255 / np.log(1 + f.max())
log_img = np.clip(c * np.log1p(f), 0, 255).astype(np.uint8) # log1p(x) = log(1 + x)
```

4.3 Power-law (gamma) transformation

$$s = c \cdot r^{\gamma}, \quad \text{where } r \in [0, 1] \text{ (normalise first)}, \quad \text{where } c = 1$$

- $\gamma < 1$ (e.g. 0.4, 0.5): curve bows upward, darks get brighter, harsh highlights get less dominant relative to midtones. Lab 2 asks for this.
- $\gamma = 1$: identity. $\gamma > 1$ (e.g. 2.2): darkens, restores washed-out images.
- Normalising to $[0, 1]$ matters: $255^{0.5} = 15.97$, which would make everything nearly black.

```
def gamma_correct(img, gamma):
    table = np.array([((i / 255.0) ** gamma) * 255 for i in
range(256)]).astype(np.uint8) # LUT
    return cv2.LUT(img, table) # works on gray or BGR

g05 = gamma_correct(gray, 0.5)
g04 = gamma_correct(gray, 0.4)
g22 = gamma_correct(gray, 2.2)
```

4.4 Piecewise-linear transformations

Contrast stretching with control points (r_1, s_1) and (r_2, s_2) :

$$s = \begin{cases} \frac{s_1}{r_1}r, & 0 \leq r < r_1 \\ \frac{s_2 - s_1}{r_2 - r_1}(r - r_1) + s_1, & r_1 \leq r \leq r_2 \\ \frac{255 - s_2}{255 - r_2}(r - r_2) + s_2, & r_2 < r \leq 255 \end{cases}$$

Special cases: $r_1 = s_1$ and $r_2 = s_2$ gives identity; $r_1 = r_2$ and $s_1 = 0, s_2 = 255$ gives a threshold; $r_1 = r_{\min}, r_2 = r_{\max}$ with $s_1 = 0, s_2 = 255$ gives min-max stretching.

```
def piecewise(img, r1, s1, r2, s2):
    r = np.arange(256, dtype=np.float32)
    table = np.interp(r, [0, r1, r2, 255], [0, s1, s2, 255]) # straight
lines between control points
    return cv2.LUT(img, np.clip(table, 0, 255).astype(np.uint8))

stretched = piecewise(gray, 70, 20, 180, 235) # middle band gets
steeper -> more contrast

# Min-max (full-range) contrast stretching
mn, mx = int(gray.min()), int(gray.max())
minmax = ((gray.astype(np.float32) - mn) / max(mx - mn, 1) *
255).astype(np.uint8)
# same thing: cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)

# Intensity-level slicing: highlight band [A, B]
A, B = 100, 160
slice_bin = np.where((gray >= A) & (gray <= B), 255, 0).astype(np.uint8)
# band white, rest black
slice_keep = np.where((gray >= A) & (gray <= B), 255, gray).astype(np.uint8)
# band white, rest unchanged
```

```
# Bit-plane slicing: plane k (0 = LSB, 7 = MSB)
planes = [((gray >> k) & 1) * 255 for k in range(8)]

negative = 255 - gray                                # or
cv2.bitwise_not(gray)
```

Plot the transfer curve to explain it (graders like this): `plt.plot(np.arange(256), table)` with labels "input r" and "output s".

4.5 Histogram processing

The normalised histogram is $p(r_k) = n_k / (M \cdot N)$. Equalization maps each level through the cumulative distribution:

$$s_k = (L-1) \frac{\sum_{j=0}^k p_r(r_j)}{\sum_{j=0}^{L-1} p_r(r_j)} = \frac{(L-1)}{MN} \sum_{j=0}^k n_j$$

Intuition: if most pixels sit in 40 to 80, the CDF rises steeply there, so those levels get spread across 0 to 255. Result: a roughly flat histogram and higher global contrast.
Downside: it also amplifies noise and can over-brighten backgrounds.

```
import matplotlib.pyplot as plt
eq = cv2.equalizeHist(gray)                                # 8-bit single
channel only
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)  #
local, limits noise boost

fig, ax = plt.subplots(2, 3, figsize=(15, 8))
for i, (im, t) in enumerate([(gray, "Raw"), (eq, "equalizeHist"), (clahe,
"CLAHE 2.0/8x8")]):
    ax[0, i].imshow(im, cmap="gray"); ax[0, i].set_title(t); ax[0,
i].axis("off")
    ax[1, i].hist(im.ravel(), bins=256, range=(0, 256), color="k")
    cdf = np.cumsum(np.bincount(im.ravel(), minlength=256)); cdf = cdf /
cdf[-1] * ax[1, i].get_ylim()[1]
    ax[1, i].plot(cdf, "r"); ax[1, i].set_title(t + " histogram + CDF")
plt.tight_layout(); plt.show()
```

Manual equalization (if told "do not use equalizeHist"):

```

hist = np.bincount(gray.ravel(), minlength=256)
cdf = hist.cumsum()
cdf_min = cdf[cdf > 0][0]
lut = np.round((cdf - cdf_min) / (cdf[-1] - cdf_min) * 255).clip(0,
255).astype(np.uint8)
eq_manual = lut[gray]                                # fancy indexing = apply LUT

```

Colour image equalization: never equalize B, G, R separately (colours shift). Equalize brightness only:

```

ycrcb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
ycrcb[:, :, 0] = cv2.equalizeHist(ycrcb[:, :, 0])    # Y = luminance
color_eq = cv2.cvtColor(ycrcb, cv2.COLOR_YCrCb2BGR)

```

Histogram calculation the OpenCV way: `hist = cv2.calcHist([gray], [0], None, [256], [0, 256])` → shape (256,1).

4.6 Colour mapping and colour balance

```

heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET)      # input 8-bit gray,
output BGR

```

JET maps low to blue, mid to green/yellow, high to red. Human eyes separate hues far better than ~50 shades of grey, so a small intensity step at a fluid boundary becomes a visible colour change.

Colour balance (gray-world): assume the average scene colour is neutral grey, so scale each channel to the common mean.

$$B' = B \cdot \frac{\bar{K}}{\bar{B}}, \quad G' = G \cdot \frac{\bar{K}}{\bar{G}}, \quad R' = R \cdot \frac{\bar{K}}{\bar{R}}, \quad \bar{K} = \frac{\bar{B} + \bar{G} + \bar{R}}{3}$$

```

def gray_world(img):
    f = img.astype(np.float32)
    means = f.reshape(-1, 3).mean(axis=0)          # per-channel means
    [B, G, R]
    f *= means.mean() / (means + 1e-6)           # broadcast over
channels
    return np.clip(f, 0, 255).astype(np.uint8)

def simple_balance(img, pct=1):
    """Stretch each channel so its pct and 100-pct percentiles hit 0 and
    255."""
    out = np.zeros_like(img)
    for ch in range(3):
        lo, hi = np.percentile(img[:, :, ch], (pct, 100 - pct))
        out[:, :, ch] = np.clip((img[:, :, ch] - lo) * 255.0 / max(hi - lo,
1), 0, 255)
    return out

balanced = gray_world(heat)

```

4.7 Task 1: chest X-ray enhancement pipeline

```

import cv2, numpy as np, matplotlib.pyplot as plt, os
os.makedirs("output", exist_ok=True)
raw = cv2.imread("data/sample_xray.png", cv2.IMREAD_GRAYSCALE)      #
relative path!
if raw is None: raise SystemExit("X-ray not found")
raw = cv2.resize(raw, (512, 512))                                     #
Kaggle images vary in size

eq      = cv2.equalizeHist(raw)                                       # 1
contrast enhancement
heat    = cv2.applyColorMap(eq, cv2.COLORMAP_JET)                   # 2
false colour
balanced = gray_world(heat)                                          # 3
colour balance
_, dense = cv2.threshold(eq, 200, 255, cv2.THRESH_BINARY)          # 4
keep only densest tissue
f = raw.astype(np.float32)
log_img = np.uint8(np.clip(255 / np.log1p(f.max()) * np.log1p(f), 0, 255))
# 5 reveal faint rib edges
gam      = gamma_correct(raw, 0.5)                                   # 6

```

```

gamma < 1 softens bone glare

steps = [(raw, "Raw"), (eq, "Equalized"), (heat, "JET"), (balanced, "Gray-
world"),
         (dense, "Threshold 200"), (log_img, "Log"), (gam, "Gamma 0.5")]
plt.figure(figsize=(20, 6))
for i, (im, t) in enumerate(steps):
    plt.subplot(1, 7, i + 1)
    plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB) if im.ndim == 3 else im,
               cmap="gray")
    plt.title(t); plt.axis("off")
    cv2.imwrite(f"output/{i}_{t.replace(' ', '_')}.png", im)
plt.tight_layout(); plt.show()

```

Analysis to write: equalization redistributes clumped dark values so lung fields and fluid become visible. JET turns subtle grey steps into hue changes. Threshold 200 isolates bone and dense fluid (tune with Otsu if unsure). Log expands the dark background to reveal faint rib edges. $\gamma = 0.5$ lifts midtones so soft lung tissue is no longer drowned by bright bone.

4.8 Task 2: CT + MRI weighted fusion

```

ct = cv2.imread("data/ct_01.png", cv2.IMREAD_GRAYSCALE)
mri = cv2.imread("data/mri_01.png", cv2.IMREAD_GRAYSCALE)
assert ct is not None and mri is not None, "check paths"
mri = cv2.resize(mri, (ct.shape[1], ct.shape[0]))          # MUST match
before blending

ct_eq, mri_eq = cv2.equalizeHist(ct), cv2.equalizeHist(mri) # independent
equalization
ct_col = cv2.cvtColor(ct_eq, cv2.COLOR_GRAY2BGR)          # CT kept grey
(edges)
mri_col = cv2.applyColorMap(mri_eq, cv2.COLORMAP_JET)      # MRI coloured
(soft tissue)

alpha, beta = 0.7, 0.3                                     # CT heavier:
sharp anatomy
fused = cv2.addWeighted(ct_col, alpha, mri_col, beta, 0)

f = fused.astype(np.float32)
fused_log = np.uint8(np.clip(255 / np.log1p(f.max()) * np.log1p(f), 0, 255))
fused_final = gamma_correct(fused_log, 1.2)                # >1 pulls back
log's brightening

show([ct_eq, mri_col, fused, fused_final], ["CT eq", "MRI JET", f"Fused a=
{alpha} b={beta}", "Log + gamma 1.2"])
for a in (0.3, 0.5, 0.7):                                  # weight
comparison, a strong extra
    print(a, "mean", cv2.addWeighted(ct_col, a, mri_col, 1 - a,
0).mean().round(1))

```

- $\alpha + \beta = 1$ keeps overall brightness in range; $\alpha + \beta > 1$ risks clipping to white.
- "Mathematically validate" = report numbers: mean and std of each image, or edge strength `cv2.Laplacian(img, cv2.CV_64F).var()` (CT high, MRI low, fused in between).
- If the two slices were not aligned, fusion would show doubled edges (ghosting); the task assumes "perfectly aligned" slices.

4.9 Task 3: real-time echocardiogram pipeline

```
import cv2, numpy as np
cap = cv2.VideoCapture("data/echo_sample.avi")          # EchoNet clips
are .avi; .mp4 works the same
if not cap.isOpened(): raise SystemExit("cannot open video")
lut_gamma = np.array([((i / 255) ** 1.5) * 255 for i in range(256)],
np.uint8)      # >1 suppresses glare
lut_log  = np.array([255 / np.log(256) * np.log1p(i) for i in range(256)],
np.uint8)
while True:
    ok, frame = cap.read()
    if not ok: break                                # end of video
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    eq  = cv2.equalizeHist(gray)                    # fight murky
contrast
    heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET)    # highlight flow
intensities
    bal  = gray_world(heat)                         # colour balance
    logd = cv2.LUT(bal, lut_log)                    # reveal dark
chambers
    out  = cv2.LUT(logd, lut_gamma)                  # suppress white
backscatter
    panel = np.hstack([frame, out])                  # raw | enhanced
(same size, 3 ch)
    panel = cv2.resize(panel, None, fx=3, fy=3)      # EchoNet frames
are 112x112, enlarge
    cv2.imshow("Raw | Enhanced", panel)
    if cv2.waitKey(30) & 0xFF == ord("q"): break    # ~33 fps; q to
quit
cap.release(); cv2.destroyAllWindows()
```

- Gamma in the echo task is set **above** 1 because the goal is to *suppress* bright backscatter. In the X-ray task it is below 1 to *lift* midtones. Know which direction does what.
- For a screenshot to `output/`: inside the loop, `if frame_no == 50:`
`cv2.imwrite("output/pipeline.png", panel)`.
- Inside Jupyter without popups, show one frame per N with Matplotlib, or save to a video with `VideoWriter` (Section 2.8).
- README for this task must say how to run it: activate the environment, `pip install -r requirements.txt`, open the notebook, run all, press `q` to stop.

5. Lab 03: Geometric transformations

Geometric transforms move pixels, $(x, y) \rightarrow (x', y')$, without changing their values; OpenCV applies affine maps with a 2×3 matrix (`warpAffine`) and projective maps with a 3×3 matrix (`warpPerspective`).

5.1 Coordinates: the trap that costs marks

- Origin (0, 0) is the **top-left** corner; x grows right, y grows **down**. A 640×480 image has corners (0, 0), (639, 0), (0, 479), (639, 479).
- NumPy indexing is `image[y, x]` (row, column). Geometric points and every OpenCV function take `(x, y)`.
- Because y points down, a positive angle in `getRotationMatrix2D` turns the picture **counter-clockwise on screen**.
- 2D transforms (image planes, photos, documents, planar objects) use 2×3 or 3×3 matrices. 3D transforms (x, y, z) use 4×4 homogeneous matrices. Mapping 3D world (X, Y, Z) to image (x, y) is camera projection, a separate topic.

5.2 Reading a 2×2 matrix: columns are the new axes

```
\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad \mathbf{A} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} a \\ c \end{bmatrix} \text{ (new x-axis)}, \quad \mathbf{A} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} b \\ d \end{bmatrix} \text{ (new y-axis)}
```

Read the matrix by **columns**: the first column is where the unit x-arrow (1, 0) lands, the second is where the unit y-arrow (0, 1) lands. Every other point follows along.

Entry	Role	Visual effect
a	x-coordinate of new x-axis	Horizontal scaling; negative flips horizontally
c	y-coordinate of new x-axis	Vertical shear (tilts vertical grid lines)
b	x-coordinate of new y-axis	Horizontal shear (tilts horizontal grid lines)
d	y-coordinate of new y-axis	Vertical scaling; negative flips vertically

The multiplication written out:

```
\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad ; \rightarrow x' = ax + by, \quad y' = cx + dy
```

5.3 The four linear building blocks

```
\underbrace{\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}}_{\text{scale}} \quad \underbrace{\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}}_{\text{rotation}} \quad \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}}_{\text{reflect across x-axis}} \quad \underbrace{\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}}_{\text{horizontal shear}}
```

- **Scaling:** values on the diagonal only, so axes stretch independently with no mixing.
- **Rotation:** all four entries change together via sin and cos, so the axes stay perpendicular and unit length. You cannot rotate by changing one entry.
- **Reflection:** a negative diagonal entry flips that axis. $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$ mirrors left-right.
- **Shear:** $x' = x + k \cdot y, y' = y$. Rows slide sideways more the further down they are.

5.4 Why linear maps cannot translate, and how homogeneous coordinates fix it

```
\mathbf{A} \mathbf{0} = \mathbf{0} \quad \text{for every } 2 \times 2 \mathbf{A}
```

The origin is pinned: multiplying (0, 0) by any 2×2 matrix gives (0, 0). A linear map can spin, stretch and shear the grid around that pin, but cannot slide it. Moving (10, 20) to (110, 70) is impossible with a 2×2 matrix alone.

Affine = linear + translation:

```
p' = \mathbf{A}p + \mathbf{b} \quad ; = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}
```

That "+b" is a separate addition step, which breaks the pure chain of matrix multiplications GPUs are optimised for. The trick: append a dummy 1 to every point, (x, y) → (x, y, 1), and fold the shift into a third column.

```
\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \\ \end{bmatrix} \; ; \rightarrow x' = x + t_x, \quad y' = y + t_y
```

The 1 multiplies t_x and t_y , producing the shift through multiplication alone. Now scale, rotation and translation are all 3×3 , so they **compose** by multiplication into one master matrix, applied once per pixel. Order matters: matrices apply right to left, and $T \cdot R \neq R \cdot T$ in general.

```
\mathbf{M}_{\text{total}} = \mathbf{T} \mathbf{R} \mathbf{S} \quad (\text{scale first, then rotate, then translate})
```

`warpAffine` takes only the top two rows of this (2×3); the bottom row of an affine matrix is always $[0, 0, 1]$.

5.5 Projective transforms (homographies)

```
\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}
```

The **denominator** is the whole difference. In affine maps the bottom row is locked to $[0, 0, 1]$, so the denominator is always 1 and scaling is uniform everywhere. In a homography the denominator changes with (x, y) : points "further away" get a bigger denominator and squeeze together. That variable division creates depth, converging lines and vanishing points.

H has 9 entries but is defined only up to scale (h_{33} is usually 1), leaving **8 degrees of freedom**, so it needs **4 point pairs** (each gives 2 equations). An affine map has 6 DOF and needs 3 pairs.

Class	DOF	Matrix	Preserves	OpenCV
Translation	2	$[I \mid t]$	Everything except position	build M by hand
Rigid (Euclidean)	3	$[R \mid t]$	Lengths, angles, area	<code>getRotationMatrix2D(c, θ, 1)</code> + shift

Class	DOF	Matrix	Preserves	OpenCV
Similarity	4	$\begin{bmatrix} sR & t \end{bmatrix}$	Angles, shape, ratios	<code>getRotationMatrix2D(c, θ, s)</code>
Affine	6	$\begin{bmatrix} A & t \end{bmatrix}$	Parallel lines, collinearity, ratios on a line	<code>getAffineTransform(3 pts)</code>
Projective	8	3×3 H	Straight lines only (not parallelism)	<code>getPerspectiveTransform(4 pts)</code>

The hierarchy (manual p.4): 2D geometric transforms split into **Affine** (Linear: rotation, scale, shear; plus Translation) and **Projective** (perspective). $\text{Rigid} \subset \text{Similarity} \subset \text{Affine} \subset \text{Projective}$.

Affine guarantees: collinear points stay collinear, parallel lines stay parallel, and the ratio $AB : BC$ along a line is unchanged. Angles and lengths may change.

5.6 Rotation (and rotation without cropping)

`getRotationMatrix2D(center, angle, scale)` returns this 2×3 matrix, with $\alpha = s \cdot \cos \theta$ and $\beta = s \cdot \sin \theta$:

$$\mathbf{M} = \begin{bmatrix} \alpha & \beta & (1-\alpha)c_x - \beta c_y \\ -\beta & \alpha & \beta c_x + (1-\alpha)c_y \end{bmatrix}$$

The third column is the translation that keeps the chosen centre fixed.

```

import cv2, numpy as np
image = cv2.imread("image.jpg")
if image is None: raise SystemExit("Error: image.jpg not found")
h, w = image.shape[:2]
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)           # 45 deg CCW, no
scaling
rotated = cv2.warpAffine(image, M, (w, h))           # dsize = (width,
height); corners cropped

# Rotate WITHOUT cropping: grow the canvas and shift the centre
cos, sin = abs(M[0, 0]), abs(M[0, 1])
nW, nH = int(h * sin + w * cos), int(h * cos + w * sin)
M[0, 2] += nW / 2 - center[0]
M[1, 2] += nH / 2 - center[1]
rotated_full = cv2.warpAffine(image, M, (nW, nH), borderValue=(255, 255,
255))
show([image, rotated, rotated_full], ["Original", "Rotate 45 (cropped)",
"Rotate 45 (full canvas)"])

```

Exact 90° steps without black corners: `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`
(also `ROTATE_180`, `ROTATE_90_COUNTERCLOCKWISE`). Mirror: `cv2.flip(img, 1)`
horizontal, `0` vertical, `-1` both.

5.7 Scaling

```

scaled = cv2.resize(image, None, fx=3, fy=3, interpolation=cv2.INTER_LINEAR)
# 3x bigger canvas
# Scaling as a matrix (same picture, canvas grown by hand):
S = np.float32([[3, 0, 0], [0, 3, 0]])
scaled_m = cv2.warpAffine(image, S, (w * 3, h * 3))
print(image.shape, "->", scaled.shape)

```

In the notebook output the 3× image looks the same size because Matplotlib fits each subplot; prove it by printing `.shape` or turning axes on.

5.8 Shear with a widened canvas

With $x' = x + k \cdot y$, the bottom row ($y = h - 1$) moves right by about $k \cdot h$, so the canvas must be $k \cdot h$ wider.

```

kx = 0.5
M = np.float32([[1, kx, 0],
                [0, 1, 0]])          # x' = x + 0.5*y
new_width = int(w + abs(kx) * h)
if kx < 0: M[0, 2] = abs(kx) * h    # negative
shear: shift right so nothing falls off the left
sheared = cv2.warpAffine(image, M, (new_width, h))

ky = 0.3                            # vertical
shear: y' = y + 0.3*x
My = np.float32([[1, 0, 0], [ky, 1, 0]])
sheared_y = cv2.warpAffine(image, My, (w, int(h + abs(ky) * w)))
show([image, sheared, sheared_y], ["Original", f"X-shear k={kx}", f"Y-shear k={ky}"])

```

5.9 Translation

```

tx, ty = 100, 50                    # right 100, down 50
M = np.float32([[1, 0, tx], [0, 1, ty]])
translated = cv2.warpAffine(image, M, (w, h))          # left/top strips
become black
# keep everything: translated_full = cv2.warpAffine(image, M, (w + tx, h + ty))

```

Negative tx moves left, negative ty moves **up** (y points down).

5.10 Rigid and similarity transforms

```

M = cv2.getRotationMatrix2D(center, 30, 1.0)          # scale MUST be 1.0 for rigid
(lengths preserved)
M[0, 2] += 100                                         # then add translation to
the existing column
M[1, 2] += 50
rigid = cv2.warpAffine(image, M, (w, h))

Ms = cv2.getRotationMatrix2D(center, 30, 0.7)         # scale != 1 -> similarity
(shape kept, size changes)
similar = cv2.warpAffine(image, Ms, (w, h))

```

Use `+=` not `=`: the third column already holds the centre-keeping offset.

5.11 General affine from 3 point pairs, and composing with homogeneous matrices

```

src = np.float32([[0, 0], [w - 1, 0], [0, h - 1]])
dst = np.float32([[0, h * 0.33], [w * 0.85, h * 0.25], [w * 0.15, h * 0.7]])
A = cv2.getAffineTransform(src, dst) # 2x3, exactly 3
pairs
affine = cv2.warpAffine(image, A, (w, h))

# Compose S then R then T as 3x3 matrices, apply once
def to3(M2x3): return np.vstack([M2x3, [0, 0, 1]])
S = np.float32([[0.5, 0, 0], [0, 0.5, 0], [0, 0, 1]])
R = to3(cv2.getRotationMatrix2D((0, 0), 20, 1.0))
T = np.float32([[1, 0, 150], [0, 1, 80], [0, 0, 1]])
M_total = (T @ R @ S)[:2] # right-to-left:
S first
combo = cv2.warpAffine(image, M_total, (w, h))

# Undo any affine
A_inv = cv2.invertAffineTransform(A)
restored = cv2.warpAffine(affine, A_inv, (w, h))

# Where does one point go? [x', y'] = M @ [x, y, 1]
print("corner (w-1, 0) ->", A @ np.array([w - 1, 0, 1]))

```

5.12 Perspective (4 points), inverse warp and document flattening

```
src_points = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h - 1]])
# TL, TR, BR, BL
dst_points = np.float32([[100, 50], [w - 150, 0], [w - 50, h - 80], [50, h -
20]])
H = cv2.getPerspectiveTransform(src_points, dst_points)      # 3x3, exactly 4
pairs, same order
perspective = cv2.warpPerspective(image, H, (w, h))

H_inv = np.linalg.inv(H)                                    # inverse
homography
back = cv2.warpPerspective(perspective, H_inv, (w, h))
# same: cv2.warpPerspective(perspective, H, (w, h),
flags=cv2.WARP_INVERSE_MAP)

pt = np.float32([[[w - 1, 0]]])                            # map points,
shape (N,1,2)
print(cv2.perspectiveTransform(pt, H))                      # does the w'
division for you
show([image, perspective, back], ["Original", "Perspective", "Inverse warp"])
```

Flatten a photographed page: put the 4 page corners (TL, TR, BR, BL) in `src`, a clean rectangle `[[0,0],[W-1,0],[W-1,H-1],[0,H-1]]` in `dst`, then `warpPerspective(img, H, (W, H))`. Full automatic version in Section 8.6.

5.13 Lab 3 quick answers

- **Why black regions after warping?** Output pixels whose source falls outside the input get `borderValue` (default 0). Change with `borderMode=cv2.BORDER_REFLECT` or `borderValue=(255,255,255)`.
- **Why does warpAffine look "backwards"?** OpenCV computes each output pixel by inverse-mapping into the source and interpolating, which avoids holes.
- **Affine vs projective in one line:** affine keeps parallel lines parallel; projective lets them meet at a vanishing point.

6. Lab 04: Feature extraction, filtering & edges

Feature extraction turns a huge pixel array into a short, meaningful vector; Lab 4 covers six histogram descriptors, convolution and its border/stride variants, and edge detection up to Canny.

6.1 Theory in one table

Idea	Meaning
Why extract features	Raw pixels are high-dimensional and noisy; features reduce dimension, keep what matters, cut computation.
Local features	From small regions: keypoints, corners, patches (matching, detection). SIFT, HOG cells, LBP codes.
Global features	Whole-image summaries: colour histogram, texture descriptors, moments.
Technique families	Histogram-based, filter-based (Gabor, Haar wavelets), deep-learning (CNN-learned).
Applications	Object, face, gesture recognition; medical anomalies; autonomous driving.
Challenges	Lighting, scale, orientation, noise.

6.2 HOG (Histogram of Oriented Gradients)

Steps: grayscale $\rightarrow$ gradients (Sobel) $\rightarrow$ magnitude and orientation per pixel $\rightarrow$ 8×8 cells $\rightarrow$ 9-bin orientation histogram per cell (0 to 180° , 20° bins, votes weighted by magnitude) $\rightarrow$ normalise over 2×2 -cell blocks (lighting robustness) $\rightarrow$ concatenate into one vector.
Classic use: pedestrian detection.

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan2(G_y, G_x)$$

Feature length for the classic 64×128 window: $7 \times 15 \text{ blocks} \times 4 \text{ cells} \times 9 \text{ bins} = \mathbf{3780}$.

```

from skimage.feature import hog
from skimage import exposure
import cv2, numpy as np, matplotlib.pyplot as plt
image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
features, hog_image = hog(image, orientations=9, pixels_per_cell=(8, 8),
                           cells_per_block=(2, 2), block_norm="L2-Hys",
                           visualize=True)
hog_vis = exposure.rescale_intensity(hog_image, in_range=(0, 10)) #
brighten for display
show([image, hog_vis], ["Original", f"HOG ({features.size} features)"])
print("HOG Feature Vector:", features.shape, features[:10])

```

Colour input: add `channel_axis=-1` (older scikit-image: `multichannel=True`). Bigger `pixels_per_cell` = shorter, coarser vector.

6.3 LBP (Local Binary Pattern)

For each pixel, compare its P neighbours on a circle of radius R with the centre: neighbour $\geq$ centre $\rightarrow 1$, else 0. Read the bits as a binary number.

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p, \quad s(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

- $P = 8, R = 1$ gives $2^8 = 256$ possible codes.
- `method='uniform'` : patterns with at most 2 bit transitions (edges, corners, spots) get their own code, all others share one, giving **$P + 2 = 10$ bins** and rotation invariance.
- The histogram of codes is the texture feature. Robust to monotonic lighting changes because it uses only comparisons.

```

from skimage import feature
image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
radius = 1; n_points = 8 * radius
lbp = feature.local_binary_pattern(image, n_points, radius, method="uniform")
hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, n_points + 3), range=
(0, n_points + 2))
hist = hist.astype("float") / (hist.sum() + 1e-6)          # normalise ->
image-size independent
plt.figure(figsize=(12, 5))
plt.subplot(121); plt.imshow(lbp, cmap="gray"); plt.title("LBP image");
plt.axis("off")
plt.subplot(122); plt.bar(range(n_points + 2), hist); plt.title("LBP
histogram")
plt.xlabel("LBP pattern"); plt.ylabel("Frequency"); plt.show()
print("LBP Feature Vector:", hist.round(3))

```

6.4 Colour histograms (1D, 2D, 3D)

Count pixels per colour bin (RGB or HSV). Global, rotation-invariant, but ignores spatial layout.

```

img = cv2.imread("image.jpg")
# 1D per channel
plt.figure(figsize=(8, 4))
for i, col in enumerate(("b", "g", "r")):
    h1 = cv2.calcHist([img], [i], None, [256], [0, 256])    # images,
channels, mask, bins, range
    plt.plot(h1, color=col)
plt.title("BGR histograms"); plt.xlim([0, 256]); plt.show()

# 2D Hue-Saturation (lighting-robust)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h2 = cv2.calcHist([hsv], [0, 1], None, [30, 32], [0, 180, 0, 256])    # hue
max is 180!
plt.imshow(h2, interpolation="nearest"); plt.xlabel("S bins"); plt.ylabel("H
bins"); plt.show()

# 3D colour histogram as one feature vector
h3 = cv2.calcHist([img], [0, 1, 2], None, [8, 8, 8], [0, 256, 0, 256, 0,
256])
feat3d = cv2.normalize(h3, h3).flatten()                    # 512-D
concat = np.concatenate([cv2.calcHist([img], [c], None, [32], [0,

```

```

256])).ravel() for c in range(3)]) # 96-D

# Compare two images' histograms (1.0 = identical for CORREL)
img2 = cv2.flip(img, 1)
h_b = cv2.normalize(cv2.calcHist([img2], [0, 1, 2], None, [8, 8, 8], [0, 256]
* 3), None).flatten()
print("similarity:", cv2.compareHist(feats3d.astype(np.float32),
h_b.astype(np.float32), cv2.HISTCMP_CORREL))

```

6.5 HED (Histogram of Edge Directions)

In this course HED = **Histogram of Edge Directions**: edge map → gradient angle at each pixel → bin the angles (e.g. 8 bins over 360°) → count. Gives the dominant edge orientations.

```

image = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
smooth = cv2.GaussianBlur(image, (5, 5), 0)
edges = cv2.Canny(smooth, 30, 70)
gx = cv2.Sobel(smooth, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(smooth, cv2.CV_64F, 0, 1, ksize=3)
angle = (np.degrees(np.arctan2(gy, gx)) + 360) % 360 # FIX: arctan2
gives -180..180
edge_angles = angle[edges > 0] # only at edge
pixels (manual uses all pixels)
hist, bin_edges = np.histogram(edge_angles, bins=8, range=(0, 360))
plt.figure(figsize=(12, 5))
plt.subplot(121); plt.imshow(edges, cmap="gray"); plt.title("Edge map")
plt.subplot(122); plt.bar(bin_edges[:-1], hist, width=45, align="edge")
plt.xticks(range(0, 361, 45)); plt.xlabel("Edge direction (deg)");
plt.title("HED"); plt.show()
print("HED Feature Vector:", hist)

```

6.6 HIG (Histogram of Intensity Gradients)

Same gradient maths, but over **every pixel** (no edge map), commonly unsigned 0 to 180° with 9 bins, optionally weighted by magnitude, then normalised. HOG is HIG computed per cell with block normalisation.

```

dx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
magnitude, orientation = cv2.cartToPolar(dx, dy, angleInDegrees=True) #
0..360
orientation = orientation % 180 #
unsigned
histogram, bins = np.histogram(orientation, bins=9, range=(0, 180),
weights=magnitude)
histogram = histogram / histogram.sum()
plt.bar(bins[:-1], histogram, width=20, align="edge"); plt.title("HIG");
plt.xlabel("Gradient orientation (deg)"); plt.show()

```

6.7 Texture energy and contrast

Over a $k \times k$ window: energy = sum of squared intensities; contrast = local standard deviation.

$$E = \sum_{(i,j) \in W} I(i,j)^2, \quad C = \sqrt{E[I^2] - (E[I])^2}$$

```

f = image.astype(np.float32) # FIX: uint8
** 2 overflows
k = 5
mean = cv2.blur(f, (k, k)) # local E[I]
mean_sq = cv2.blur(f * f, (k, k)) # local
E[I^2]
energy = mean_sq * k * k # local sum
of squares
contrast = np.sqrt(np.maximum(mean_sq - mean ** 2, 0)) # local std
e_hist, _ = np.histogram(energy, bins=64, range=(0, energy.max()))
c_hist, _ = np.histogram(contrast, bins=64, range=(0, contrast.max()))
plt.figure(figsize=(12, 4))
plt.subplot(121); plt.plot(e_hist / e_hist.sum(), "b"); plt.title("Texture
energy histogram")
plt.subplot(122); plt.plot(c_hist / c_hist.sum(), "r"); plt.title("Texture
contrast histogram")
plt.tight_layout(); plt.show()

```

Smooth, uniform textures (plastic, polished metal) → low contrast. Rough textures (bark, gravel, knitted fabric) → high contrast.

6.8 Lab 4 task: texture analysis for material classification

Model answer structure: describe two techniques, what textures suit each, then pros and cons.

	LBP	Texture energy / contrast (or GLCM)
How it works	Binary code from neighbour-vs-centre comparisons; histogram of codes	Local sum of squares and local std in a sliding window; histogram of each
Best for	Fine micro-patterns: fabric weave, wood grain, brushed metal	Coarse roughness differences: polished vs rough metal, smooth vs rough wood
Strengths	Lighting-invariant, fast, rotation-invariant (uniform), small 10-D vector	Very simple, intuitive, cheap with box filters
Limitations	Noise-sensitive at R = 1, misses large-scale structure (use several radii)	Sensitive to brightness and scale; no direction information

Add HOG if shapes matter, colour histograms if materials differ in colour (but colour fails across paint finishes).

```

from skimage.feature import local_binary_pattern
def texture_vector(gray):
    lbp = local_binary_pattern(gray, 8, 1, "uniform")
    h, _ = np.histogram(lbp, bins=np.arange(11), range=(0, 10)); h = h /
h.sum()
    f = gray.astype(np.float32)
    std = np.sqrt(np.maximum(cv2.blur(f * f, (5, 5)) - cv2.blur(f, (5, 5)) **
2, 0))
    return np.concatenate([h, [std.mean() / 255, std.std() / 255]]) # 12-D

# training: one or more patches per material
train = {name: texture_vector(cv2.imread(f"data/{name}.jpg", 0)) for name in
("wood", "metal", "fabric")}
test = texture_vector(cv2.imread("data/unknown.jpg", 0))
pred = min(train, key=lambda n: np.linalg.norm(train[n] - test)) #
nearest neighbour
print("Predicted material:", pred)

```

6.9 Filtering and convolution

Slide a small kernel over the image; each output pixel = sum of kernel $\times$ neighbourhood (element-wise products, then summed).

$$g(x,y) = \sum_{i=-a}^a \sum_{j=-b}^b w(i,j) \cdot f(x+i, y+j)$$

Strictly that is correlation; true convolution flips the kernel first. `cv2.filter2D` correlates; for symmetric kernels the result is identical.

```
image = cv2.imread("image.jpg")
box    = cv2.filter2D(image, -1, np.ones((3, 3), np.float32) / 9)      #
mean of 9 neighbours; -1 = same depth
box2   = cv2.blur(image, (3, 3))                                       #
identical
g1d     = cv2.getGaussianKernel(5, 1.0)                                #
5x1 column!
gauss   = cv2.filter2D(image, -1, g1d @ g1d.T)                         #
FIX: outer product = 5x5
gauss2  = cv2.GaussianBlur(image, (5, 5), 1.0)                         #
identical, easier
median  = cv2.medianBlur(image, 5)                                     #
best for salt-and-pepper
bilat   = cv2.bilateralFilter(image, 9, 75, 75)                       #
smooths, keeps edges
sharpen = cv2.filter2D(image, -1, np.array([[0, -1, 0], [-1, 5, -1], [0, -1,
0]], np.float32))
emboss  = cv2.filter2D(image, -1, np.array([[-2, -1, 0], [-1, 1, 1], [0, 1,
2]], np.float32))
```

The manual applies `getGaussianKernel` directly in `filter2D`; that kernel is 5×1 , so it blurs only vertically. Use `g @ g.T` or `cv2.sepFilter2D(image, -1, g, g)`.

Kernel	Matrix	Effect
Box 3×3	all $1/9$	Average, blur
Gaussian 3×3	$[1\ 2\ 1; 2\ 4\ 2; 1\ 2\ 1] / 16$	Smooth blur, less blocky
Sobel Gx	$[-1\ 0\ 1; -2\ 0\ 2; -1\ 0\ 1]$	Vertical edges (horizontal change)
Sobel Gy	$[-1\ -2\ -1; 0\ 0\ 0; 1\ 2\ 1]$	Horizontal edges (vertical change)
Prewitt Gx	$[-1\ 0\ 1; -1\ 0\ 1; -1\ 0\ 1]$	Like Sobel, no centre weighting

Kernel	Matrix	Effect
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Second derivative, all directions
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Identity + Laplacian-style boost

6.10 Convolution types: standard, valid, same, strided (correct versions)

Output size for input n , kernel k , padding p , stride s :

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

$\text{valid: } p=0 \rightarrow n-k+1$,
 $\text{same: } p=\lceil \frac{k-1}{2} \rceil$,
 $s=1 \rightarrow n$

```

gray = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)
kernel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]], np.float32)
k = kernel.shape[0]; p = k // 2

standard = cv2.filter2D(gray, -1, kernel) #
default border = REFLECT_101
same_zero = cv2.filter2D(gray, -1, kernel, borderType=cv2.BORDER_CONSTANT)
# TRUE zero padding, same size
valid = same_zero[p:-p, p:-p] #
TRUE valid: (H-k+1, W-k+1)
strided = valid[:,::2, ::2] #
stride 2 (filter2D has no stride arg)
print(gray.shape, valid.shape, strided.shape)

# From scratch (if asked to implement convolution manually)
def conv2d(img, ker, stride=1, pad=0):
    ker = np.flipud(np.fliplr(ker)) # true convolution
    # flips the kernel
    img = np.pad(img, pad, mode="constant") # zero padding
    kh, kw = ker.shape
    oh = (img.shape[0] - kh) // stride + 1
    ow = (img.shape[1] - kw) // stride + 1
    out = np.zeros((oh, ow), np.float32)
    for i in range(oh):
        for j in range(ow):
            r, c = i * stride, j * stride
            out[i, j] = np.sum(img[r:r + kh, c:c + kw] * ker)
    return out

```

Display any signed/float result with `cv2.convertScaleAbs(result)` (absolute value, clipped to uint8).

6.11 Edge detection operators

Method	Order	Notes
Sobel	1st derivative	Gx, Gy with centre weight 2; simple; thick edges; noise-sensitive
Scharr	1st	More accurate rotation behaviour for 3×3
Prewitt	1st	Equal weights; no OpenCV function, use <code>filter2D</code>
Laplacian	2nd	Rotation-invariant; very noise-sensitive; double edges
LoG (Marr-Hildreth)	2nd	Gaussian blur then Laplacian; edges at zero crossings
Canny	1st + logic	Blur, gradient, NMS, hysteresis; thin, connected edges
CNN-based	learned	Adapts to domain (e.g. Holistically-Nested Edge Detection)

```

gray = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)           # CV_64F keeps
negative gradients
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.uint8(np.clip(np.sqrt(sx ** 2 + sy ** 2), 0, 255))
scx = cv2.Scharr(gray, cv2.CV_64F, 1, 0)
px = cv2.filter2D(gray.astype(np.float32), -1, np.array([[ -1, 0, 1]] * 3,
np.float32))      # Prewitt x
py = cv2.filter2D(gray.astype(np.float32), -1, np.array([[ -1] * 3, [0] * 3,
[1] * 3], np.float32))
prewitt = cv2.convertScaleAbs(np.sqrt(px ** 2 + py ** 2))
lap = cv2.convertScaleAbs(cv2.Laplacian(gray, cv2.CV_64F, ksize=3))
log = cv2.convertScaleAbs(cv2.Laplacian(cv2.GaussianBlur(gray, (5, 5), 1.4),
cv2.CV_64F))
edges_bin = (np.sqrt(sx ** 2 + sy ** 2) > 100).astype(np.uint8) * 255      #
threshold the magnitude
show([gray, cv2.convertScaleAbs(sx), cv2.convertScaleAbs(sy), mag, prewitt,
lap, log, edges_bin],
      ["Gray", "Sobel X", "Sobel Y", "Magnitude", "Prewitt", "Laplacian",
"LoG", "|G| > 100"], cols=4)

```

Why `CV_64F` ? A dark-to-bright edge gives a positive gradient, bright-to-dark a negative one. In `uint8` the negatives clip to 0 and half the edges vanish.

6.12 Canny in four stages

1. **Gaussian smoothing:** remove noise so it does not create false edges. Larger σ = fewer, smoother edges, lost fine detail.
2. **Gradient:** Sobel $G_x, G_y \rightarrow$ magnitude and direction per pixel.
3. **Non-maximum suppression:** keep a pixel only if it is the peak along its gradient direction (compare with the two neighbours across the edge). Thick ridges become 1-pixel lines.
4. **Hysteresis (double threshold):** above T_{high} = strong edge (kept). Below T_{low} = discarded. Between = weak; kept **only if connected** to a strong edge.

$$|\nabla I| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan \frac{G_y}{G_x}$$

```

image = cv2.imread("image.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
edges = cv2.Canny(blurred, 50, 150)           # low, high (ratio 1:2 to
1:3 is the rule of thumb)
edges_L2 = cv2.Canny(blurred, 50, 150, apertureSize=3, L2gradient=True)  #
exact sqrt magnitude
v = np.median(blurred)                        # auto thresholds from the
median
auto = cv2.Canny(blurred, int(max(0, 0.67 * v)), int(min(255, 1.33 * v)))
show([image, gray, blurred, edges, auto], ["Original", "Gray", "Blur 5x5
s=1.4", "Canny 50/150", "Auto Canny"])

```

Sobel vs Canny (manual's comparison): Sobel = one gradient step, thick and noisy edges.
 Canny = full pipeline, thin, connected, noise-robust edges.

7. Lab 05: Image segmentation (all 10 tasks)

Segmentation splits an image into non-overlapping regions that share colour, intensity or texture; every Lab 5 task must show the original and the result, record and justify parameters, save the output, and use no prebuilt segmentation library or deep model.

7.1 Methods at a glance

Method	Assumption about the image	Key parameter	Fails when
Global threshold	One intensity separates object from background everywhere	T	Lighting varies across the image
Adaptive threshold	Each small neighbourhood has roughly even lighting	blockSize, C	Block too small (noise) or too big (acts global)
Otsu	Histogram is bimodal (two peaks)	none (automatic)	One class tiny, or three+ classes
Colour (HSV <code>inRange</code>)	Target has a distinctive hue	H, S, V bounds	Hue shared with background, shadows, highlights

Method	Assumption about the image	Key parameter	Fails when
Canny (hysteresis)	Boundaries are sharp intensity changes	low, high	Low-contrast or textured regions; gives edges, not regions
Region growing	Target is connected and roughly uniform	seed, tolerance	Leaks through weak boundaries; seed-dependent
Watershed	Objects are blob-like; touching ones have a "neck"	distance-transform threshold	Too low merges, too high over-splits or loses objects
K-means	Regions differ in colour; K roughly known	K	Same colour, different objects; random init

Segmentation maths in the manual: thresholding $S(x, y) = \text{object}$ if $I(x, y) > T$, else background; K-means minimises the within-cluster sum of squared distances; graph methods cut a pixel graph at minimum total edge weight.

$$S(x,y) = \begin{cases} \text{object} & I(x,y) > T \\ \text{background} & \text{otherwise} \end{cases} \quad \text{and} \quad J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Shared setup for every task block below:

```
import cv2, numpy as np, matplotlib.pyplot as plt, os
os.makedirs("output", exist_ok=True)
# show() from Section 2.1 must be defined
```

7.2 Task 1: uneven lighting, global vs adaptive (sudoku.png)

```
gray = cv2.imread("sudoku.png", cv2.IMREAD_GRAYSCALE)
if gray is None: raise SystemExit("sudoku.png not found")
T = [60, 100, 140]
globs = [cv2.threshold(gray, t, 255, cv2.THRESH_BINARY)[1] for t in T]
adaptive = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY, 11, 2)
show([gray] + globs + [adaptive], ["Original"] + [f"Global T={t}" for t in T]
+ ["Adaptive Gauss 11/2"], cols=5)
cv2.imwrite("output/task1_adaptive.png", adaptive)

# Where does global fail? Compare mean brightness of the four quadrants
h, w = gray.shape
for name, q in {"TL": gray[:h//2, :w//2], "TR": gray[:h//2, w//2:], "BL":
gray[h//2:, :w//2], "BR": gray[h//2:, w//2:]}.items():
    print(name, "mean brightness", q.mean().round(1))
```

Parameters: T = 60, 100, 140 bracket the histogram's middle. blockSize 11 $\approx$ one character stroke region; C = 2 removes faint background noise.

Answer: a single T compares every pixel against one number. Where illumination is high, background paper exceeds T and stays white, but dark text in shadowed regions also falls below the background level, so a low T loses characters in shadow and a high T turns whole shadowed areas black. The darkest quadrant (lowest mean above) is where global thresholding fails. Adaptive thresholding computes a separate T for each pixel from its own neighbourhood, so it follows the lighting gradient.

7.3 Task 2: adaptive blockSize, C and method grid

```
blocks, Cs = [5, 15, 51], [2, 5, 10]
methods = {"Mean": cv2.ADAPTIVE_THRESH_MEAN_C, "Gauss":
cv2.ADAPTIVE_THRESH_GAUSSIAN_C}
fig, ax = plt.subplots(len(methods) * len(blocks), len(Cs), figsize=(12, 22))
for mi, (mname, m) in enumerate(methods.items()):
    for bi, b in enumerate(blocks):
        for ci, c in enumerate(Cs):
            res = cv2.adaptiveThreshold(gray, 255, m, cv2.THRESH_BINARY, b,
c)

            a = ax[mi * len(blocks) + bi, ci]
            a.imshow(res, cmap="gray"); a.set_title(f"{mname} b={b} C={c}",
fontsize=9); a.axis("off")
plt.tight_layout(); plt.savefig("output/task2_grid.png", dpi=120); plt.show()
```

The threshold for each pixel is the local (mean or Gaussian-weighted mean) minus C:

$$T(x,y) = \text{operatorname{mean}}_{b \times b}(x,y) - C$$

Question	Answer
Neighbourhood too small (b = 5)	Window sits inside a stroke or flat paper, so tiny noise is treated as detail: speckle, hollow or broken characters.
Neighbourhood too large (b = 51)	Window spans several lighting levels, so it behaves like global thresholding again: shadows return, thin lines thicken or merge.
Increasing C	Lowers every local threshold, so fewer pixels count as dark: less background noise but thin or faint strokes break. Lower C = more noise.
Cleanest combination	Typically Gaussian, b = 11 to 15, C = 2 to 5 for sudoku.png; pick from your grid and say why.
Better method	Gaussian usually: nearby pixels weigh more, so edges are smoother and less blocky noise. Mean is slightly faster.

Rules: blockSize must be **odd and > 1**; input must be 8-bit single channel.

7.4 Task 3: Otsu on coins, with contrast changes

Otsu tries every T and picks the one that maximises between-class variance (equivalently minimises within-class variance):

$$T^{\{*\}} = \arg\max_{\{T\}}; \sigma_B^2(T) = \omega_0(T)\omega_1(T)\big[\mu_0(T) - \mu_1(T)\big]^2$$

```
img = cv2.imread("coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
variants = {
    "original": gray,
    "contrast x1.5": cv2.convertScaleAbs(gray, alpha=1.5, beta=0), # new =
clip(1.5*old)
    "brighter +40": cv2.convertScaleAbs(gray, alpha=1.0, beta=40),
    "blurred 5x5": cv2.GaussianBlur(gray, (5, 5), 0),
    "equalized": cv2.equalizeHist(gray),
}
for name, g in variants.items():
    T, mask = cv2.threshold(g, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# 0 is ignored
    print(f"{name:15s} Otsu T = {T:.0f}")
    variants[name] = (g, mask, T)

g, mask, T = variants["original"]
plt.figure(figsize=(15, 4))
plt.subplot(131); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));
plt.title("Original"); plt.axis("off")
plt.subplot(132); plt.hist(g.ravel(), 256, (0, 256), color="gray");
plt.axvline(T, color="r"); plt.title(f"Histogram, Otsu T={T:.0f}")
plt.subplot(133); plt.imshow(mask, cmap="gray"); plt.title("Otsu mask");
plt.axis("off")
plt.tight_layout(); plt.savefig("output/task3_otsu.png"); plt.show()
```

Answer: Otsu reads the histogram and finds the valley between the two peaks automatically, so the same code adapts to each new image. When contrast or brightness change, the peaks move and Otsu's T moves with them (record the numbers above), while a fixed manual T would break. It relies on a bimodal histogram; blur first if noise blurs the valley. Objects darker than the background? Use `THRESH_BINARY_INV + THRESH_OTSU` to make them white.

7.5 Task 4: HSV colour segmentation (too-restrictive vs final mask)

HSV separates colour (Hue) from strength (Saturation) and brightness (Value), so a shadow changes V but barely changes H. OpenCV ranges: **H 0 to 179**, S 0 to 255, V 0 to 255.

Colour	H range (OpenCV)
Red	0 to 10 and 170 to 179 (wraps around)
Orange	10 to 22
Yellow	22 to 35
Green	35 to 85
Cyan / light blue	85 to 100
Blue	100 to 130
Purple / magenta	130 to 170

```
img = cv2.imread("smarties.png")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
print("HSV at (x=100, y=80):", hsv[80, 100])           # probe a pixel you
                                                         know is the target

restrictive = cv2.inRange(hsv, np.array([25, 200, 200]), np.array([30, 255,
255])) # too narrow
final       = cv2.inRange(hsv, np.array([20, 100, 100]), np.array([35, 255,
255])) # yellow
final = cv2.morphologyEx(final, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
# remove specks
final = cv2.morphologyEx(final, cv2.MORPH_CLOSE, np.ones((5, 5), np.uint8))
# fill small holes
extracted = cv2.bitwise_and(img, img, mask=final)

# Red needs TWO ranges OR-ed together
red = cv2.inRange(hsv, (0, 100, 80), (10, 255, 255)) | cv2.inRange(hsv, (170,
100, 80), (179, 255, 255))

show([img, restrictive, final, extracted], ["Original", "Restrictive H25-30
S,V>=200", "Final H20-35 S,V>=100", "Extracted"])
cv2.imwrite("output/task4_extracted.png", extracted)
```

Why the restrictive mask fails: a real object is not one colour. Curved surfaces have shading (lower V) and specular highlights (lower S, higher V), and the camera adds noise. A narrow H band with $S, V \geq 200$ keeps only the brightest, most saturated pixels, so the mask is patchy with holes and missing edges. Widening S and V lower bounds to about 100 accepts the shaded parts while H still rejects other colours.

7.6 Task 5: Canny with three threshold pairs

```
gray = cv2.cvtColor(cv2.imread("smarties.png"), cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
pairs = [(50, 100), (50, 200), (150, 300)]          # 1 & 2: high rises, low
                                                    # fixed; 3: both high
res = [cv2.Canny(blur, lo, hi) for lo, hi in pairs]
show([gray] + res, ["Original"] + [f"Canny {lo}/{hi}" for lo, hi in pairs])
for (lo, hi), e in zip(pairs, res):
    print(f"{lo}/{hi}: edge pixels = {np.count_nonzero(e)}")
    cv2.imwrite(f"output/task5_canny_{lo}_{hi}.png", e)

# Visualise strong vs weak candidates (before linking), useful for the
# "identify" question
mag = np.hypot(cv2.Sobel(blur, cv2.CV_64F, 1, 0), cv2.Sobel(blur, cv2.CV_64F,
0, 1))
lo, hi = 50, 150
vis = np.zeros((*gray.shape, 3), np.uint8)
vis[(mag >= lo) & (mag < hi)] = (0, 255, 255)      # weak candidates:
yellow (BGR)
vis[mag >= hi] = (0, 0, 255)                       # strong: red
show([vis], ["red = strong, yellow = weak candidates"])
```

Note that `cv2.Canny` uses an unnormalised Sobel magnitude, so the manual-magnitude picture is an approximation of its internal values.

Term	Where you see it
Strong edges	Outer outlines of objects; survive all three pairs.
Weak edges	Faint inner shading lines; present in 50/100, gone in 150/300.
Missing edges	Low-contrast boundaries (two similar colours touching) vanish when high is raised.

Term	Where you see it
Unwanted edges	Texture, noise and highlights appear when low is small.

Explanation: high decides which pixels **start** an edge: raising it (50/100 $\rightarrow$ 50/200) removes weaker outlines that had no strong pixel. Low decides how far an edge can **extend**: lowering it lets edges continue through faint parts (better continuity, more noise). Raising both (150/300) keeps only the strongest boundaries.

7.7 Task 6: region growing on a brain MRI (from scratch)

```

from collections import deque

def region_growing(img, seed, thresh, connectivity=4):
    """seed = (x, y). Adds neighbours whose |I - I_seed| < thresh. Returns
    uint8 mask."""
    h, w = img.shape
    mask = np.zeros((h, w), np.uint8)
    visited = np.zeros((h, w), bool)
    sx, sy = seed
    seed_val = int(img[sy, sx])                # NumPy is [row=y,
col=x]
    q = deque([(sx, sy)]); visited[sy, sx] = True
    nbrs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    if connectivity == 8: nbrs += [(1, 1), (1, -1), (-1, 1), (-1, -1)]
    while q:
        x, y = q.popleft()
        if abs(int(img[y, x]) - seed_val) < thresh:
            mask[y, x] = 255
            for dx, dy in nbrs:
                nx, ny = x + dx, y + dy
                if 0 <= nx < w and 0 <= ny < h and not visited[ny, nx]:
                    visited[ny, nx] = True; q.append((nx, ny))
    return mask

mri = cv2.imread("brain_mri.png", cv2.IMREAD_GRAYSCALE)
mri = cv2.GaussianBlur(mri, (5, 5), 0)        # smoothing reduces
leaks through noise
h, w = mri.shape
seeds = [(w // 2, h // 2), (w // 3, h // 2)] # change to a point
inside your target

```

```

thresholds = [10, 25, 50]
results, titles = [mri], ["MRI"]
for s in seeds:
    for t in thresholds:
        m = region_growing(mri, s, t)
        results.append(m); titles.append(f"seed {s}, T={t},
{np.count_nonzero(m)} px")
show(results, titles, cols=4)
m_over = cv2.cvtColor(mri, cv2.COLOR_GRAY2BGR); m_over[results[2] > 0] = (0,
0, 255) # overlay in red
cv2.imwrite("output/task6_region.png", m_over)

```

Pick a seed by clicking: `plt.imshow(mri, cmap="gray"); plt.show()` with axes on, read the (x,y) of the target, or use `np.unravel_index(np.argmax(mri), mri.shape)` (returns (y, x)) to seed on the brightest pixel.

Why the seed changes everything: the region is defined as "pixels connected to the seed whose intensity is close to the seed's intensity". A different seed has a different reference value (so a different set of pixels qualifies) and starts in a different connected area (so it can only spread through its own neighbours). Seeding in grey matter versus white matter, or in a lesion, yields entirely different masks. Threshold too small → region stops early (under-segmentation); too large → leaks across weak boundaries into neighbouring tissue.

Variant: compare with the **running region mean** instead of the seed value for smoother growth (update the mean as pixels are added).

7.8 Task 7: marker-based watershed for touching coins

```

img = cv2.imread("coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5, 5), 0) #
1 preprocessing
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU) # 2 thresholding
if np.mean(np.r_[thresh[0], thresh[-1], thresh[:, 0], thresh[:, -1]]) > 127:
    thresh = cv2.bitwise_not(thresh) #
coins must be WHITE
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2) # 3
noise removal
sure_bg = cv2.dilate(opening, kernel, iterations=3) #

```

```

4 sure background
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5) #
5 distance transform
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0) #
6 sure foreground
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg) #
7 unknown region
n, markers = cv2.connectedComponents(sure_fg) #
8 labels 0..n-1
markers = markers + 1 #
background = 1, not 0
markers[unknown == 255] = 0 #
0 = let watershed decide
markers = cv2.watershed(img, markers) #
9 needs 3-channel 8-bit image
out = img.copy(); out[markers == -1] = (0, 0, 255) #
10 boundaries in RED (BGR)
count = len([l for l in np.unique(markers) if l > 1])
print("foreground markers:", n - 1, "| separated coins:", count)
show([img, thresh, sure_bg, dist, sure_fg, unknown, out],
      ["Original", "Threshold", "Sure BG", "Distance transform", "Sure FG",
       "Unknown", f"Watershed: {count} coins"], cols=4)
cv2.imwrite("output/task7_watershed.png", out)

```

How it works: the distance transform gives each white pixel its distance to the nearest black pixel, so each coin's centre becomes a peak even where coins touch. Thresholding those peaks gives one seed per coin (sure foreground). Dilating gives a region surely outside the coins (sure background). The ring between them is unknown. Watershed floods from the labelled seeds and builds a dam (label -1) where floods from two seeds meet: the dividing line between touching coins.

After `watershed`: -1 = boundary, 1 = background, 2..n = individual objects.

7.9 Task 8: tuning the distance-transform threshold

```

def watershed_count(img, frac):
    gray = cv2.GaussianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), (5, 5), 0)
    _, th = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
    if np.mean(np.r_[th[0], th[-1], th[:, 0], th[:, -1]]) > 127: th =
cv2.bitwise_not(th)

```

```

k = np.ones((3, 3), np.uint8)
op = cv2.morphologyEx(th, cv2.MORPH_OPEN, k, iterations=2)
bg = cv2.dilate(op, k, iterations=3)
d = cv2.distanceTransform(op, cv2.DIST_L2, 5)
_, fg = cv2.threshold(d, frac * d.max(), 255, 0); fg = np.uint8(fg)
n, mk = cv2.connectedComponents(fg); mk = mk + 1; mk[cv2.subtract(bg, fg)
== 255] = 0
mk = cv2.watershed(img.copy(), mk)
out = img.copy(); out[mk == -1] = (0, 0, 255)
return n - 1, len([1 for l in np.unique(mk) if l > 1]), out

fracs = [0.1, 0.3, 0.5, 0.7, 0.9]
rows, outs = [], []
for f in fracs:
    nfg, nreg, out = watershed_count(img, f)
    rows.append((f, nfg, nreg)); outs.append(out)
show(outs, [f"{f} x max: {a} markers, {b} regions" for f, a, b in rows],
cols=3)
import pandas as pd
print(pd.DataFrame(rows, columns=["Distance threshold (x max)", "Foreground
markers", "Regions"]).to_string(index=False))

```

Fill the "Observed result" column yourself from the pictures. Typical pattern:

Threshold	Effect
Low (0.1 to 0.2)	Sure-foreground blobs still touch, so touching coins share one marker: merged .
Middle (0.4 to 0.6)	One compact marker per coin: best separation . This is the range to report.
High (0.8 to 0.9)	Small or less-round coins lose their peak entirely: missed objects ; irregular shapes may split into two peaks: over-segmentation .

`cv2.distanceTransform` arguments: `DIST_L2` = Euclidean, mask size 5 (3 is faster and coarser, `DIST_MASK_PRECISE` exact).

7.10 Task 9: K-means colour segmentation (K = 2, 4, 6)

```
img = cv2.imread("dog.jpeg")
pixels = np.float32(img.reshape(-1, 3))          # (N, 3) feature
vectors, float32 required
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)  #
max 100 iter or move < 0.2
results, titles = [img], ["Original"]
for K in (2, 4, 6):
    compactness, labels, centers = cv2.kmeans(pixels, K, None, criteria, 10,
cv2.KMEANS_PP_CENTERS)
    #                                     bestLabels
attempts=10
    seg = np.uint8(centers[labels.flatten()]).reshape(img.shape)      #
each pixel -> its centre colour
    counts = np.bincount(labels.flatten(), minlength=K) / labels.size
    print(f"K={K}: compactness={compactness:.3e}")
    for c, p in zip(np.uint8(centers), counts): print(f"    centre BGR
{tuple(int(v) for v in c)} covers {p:.1%}")
    cv2.imwrite(f"output/task9_k{K}.png", seg)
    results.append(seg); titles.append(f"K={K}")
show(results, titles)
```

- K = 2: image collapses to light vs dark (foreground vs background silhouette); heavy simplification.
- K = 4: main regions appear (fur, background, shadows, highlights); moderate simplification.
- K = 6: close to the original look with posterised flat colours; least simplification.
- **Compactness** (sum of squared distances to centres) drops as K rises; the "elbow" where it stops dropping sharply suggests a good K.
- Better grouping by colour regardless of brightness: cluster in Lab or HSV (`cv2.cvtColor` first, convert back for display). Add pixel coordinates as extra features to make clusters spatially compact.

7.11 Task 10: segmentation engineer, three methods on one image

Choose smarties.png (coloured, touching objects, uneven background): it lets each method show a strength and a weakness.

```

img = cv2.imread("smarties.png")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Method 1: Otsu
_, m1 = cv2.threshold(cv2.GaussianBlur(gray, (5, 5), 0), 0, 255,
cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
# Method 2: HSV saturation mask (colourful candies vs pale background)
m2 = cv2.inRange(hsv, (0, 80, 50), (179, 255, 255))
m2 = cv2.morphologyEx(m2, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
# Method 3: K-means, K = 4
px = np.float32(img.reshape(-1, 3))
_, lab, cen = cv2.kmeans(px, 4, None, (cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2), 10, cv2.KMEANS_PP_CENTERS)
m3 = np.uint8(cen)[lab.flatten()].reshape(img.shape)

for name, m in [("Otsu", m1), ("HSV", m2)]:
    n, _ = cv2.connectedComponents(m); print(name, "objects found:", n - 1)
show([img, m1, m2, m3], ["Original", "Method 1: Otsu", "Method 2: HSV S>=80",
"Method 3: K-means K=4"])
cv2.imwrite("output/task10_compare.png", np.hstack([img, cv2.cvtColor(m1,
cv2.COLOR_GRAY2BGR), cv2.cvtColor(m2, cv2.COLOR_GRAY2BGR), m3]))

```

Write this table from **your** outputs (the cells below are the expected pattern):

	Otsu	HSV colour	K-means
Assumption	Bimodal grey histogram	Objects are more saturated than background	Regions differ in colour; K known
Succeeds on	Dark/strong-coloured candies	All coloured candies, incl. ones similar in grey	Groups candies by colour family
Fails on	Yellow/light candies (grey close to background), shadows	Pale or low-saturation items, specular highlights	Shadows become their own cluster; no object separation
Most sensitive parameter	Pre-blur size	S lower bound	K

Conclusion template: "HSV produced the most meaningful regions because the defining property of this image is colour saturation, not brightness: several candies have the same grey level as the background, which breaks Otsu's bimodal assumption, while K-means groups by colour family but cannot tell object from shadow. To count touching candies, the HSV mask would feed into watershed (Task 7)."

7.12 Morphology cheat (used everywhere above)

Op	Code	Effect
Erode	<code>cv2.erode(m, k, iterations=1)</code>	Shrinks white, removes specks
Dilate	<code>cv2.dilate(m, k, iterations=1)</code>	Grows white, fills gaps
Opening	<code>cv2.morphologyEx(m, cv2.MORPH_OPEN, k)</code>	Erode then dilate: removes small noise
Closing	<code>cv2.morphologyEx(m, cv2.MORPH_CLOSE, k)</code>	Dilate then erode: fills small holes
Gradient	<code>cv2.morphologyEx(m, cv2.MORPH_GRADIENT, k)</code>	Outline of shapes

Round kernel: `k = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))`.

8. Lab 06: Wavelets, Hough transform, SIFT & applications

The sixth manual (titled "Lab 05: Wavelets, Boundary Detection, Hough Transform & SIFT") pairs with the Lab 06 task sheet: wavelets for signals, Hough voting for lines and circles, and SIFT matching with RANSAC homography for recognition and stitching.

8.1 Wavelet transforms

A wavelet is a short, localised wave. Sliding it across a signal at different **scales** tells you which frequencies occur **when**, which a Fourier transform cannot (Fourier gives frequencies for the whole signal at once). That makes wavelets ideal for non-stationary signals: spikes, bursts, sudden faults.

Type	What it does	Families	Uses
CWT (continuous)	Every scale and shift; redundant, detailed time- frequency map	Morlet (<code>mor1</code>), Mexican hat	Seismic, oscillation analysis
DWT (discrete)	Halves the signal into approximation (low freq) + detail (high freq), repeat on approximation	Haar, Daubechies (<code>db4</code>), Symlets	Compression, denoising, features
WPT (packet)	Splits both approximation and detail at each level	same as DWT	Complex frequency content, ML features
Inverse (IDWT)	Rebuilds the signal from coefficients		Reconstruction after denoising

Denoising idea: noise lives in small detail coefficients; real structure lives in large ones. Shrink small details toward zero (soft threshold) and reconstruct. Universal threshold, with σ estimated from the finest details:

$$\lambda = \sigma \sqrt{2 \ln N}, \quad \sigma = \frac{\operatorname{median}(|d_1|)}{0.6745}, \quad \text{soft}(d) = \operatorname{sign}(d) \cdot \max(|d| - \lambda, 0)$$

```
import pywt, numpy as np, matplotlib.pyplot as plt
t = np.linspace(0, 1, 1024)
signal = np.sin(2 * np.pi * 5 * t) + 0.3 * np.random.randn(t.size)

cA, cD = pywt.dwt(signal, "db4") # one level:
approximation, detail
coeffs = pywt.wavedec(signal, "db4", level=4) # [cA4, cD4, cD3,
cD2, cD1]
sigma = np.median(np.abs(coeffs[-1])) / 0.6745
lam = sigma * np.sqrt(2 * np.log(signal.size))
den_coeffs = [coeffs[0]] + [pywt.threshold(c, lam, mode="soft") for c in
coeffs[1:]]
denoised = pywt.waverec(den_coeffs, "db4")[:signal.size] # inverse DWT
(trim padding)

cwt_mat, freqs = pywt.cwt(signal, np.arange(1, 64), "mor1") # CWT scalogram
(scales x time)
```

```

wp = pywt.WaveletPacket(signal, "db4", maxlevel=3)          # WPT
leaves = [n.path for n in wp.get_level(3, "natural")]      # 8 sub-bands:
'aaa', 'aad', ...

fig, ax = plt.subplots(3, 1, figsize=(12, 8))
ax[0].plot(signal, label="noisy"); ax[0].plot(denoised, label="denoised");
ax[0].legend()
ax[1].imshow(np.abs(cwt_mat), aspect="auto", cmap="jet");
ax[1].set_title("CWT scalogram (Morlet)")
ax[2].plot(cD); ax[2].set_title("Level-1 detail coefficients")
plt.tight_layout(); plt.show()

# 2-D DWT on an image: LL (approx), LH, HL, HH (details)
import cv2
gray = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE).astype(np.float32)
LL, (LH, HL, HH) = pywt.dwt2(gray, "haar")

```

Audio denoising (the manual's figure): load with `scipy.io.wavfile.read`, add noise, apply the same wavedec → threshold → waverec, compare the three waveforms.

8.2 Boundary detection recap

Sobel, Canny and LoG code is in Section 6.11 and 6.12. MCQ points from this manual:

- Sobel uses two 3×3 kernels; magnitude = edge strength, direction = edge orientation. G_x responds to vertical edges, G_y to horizontal edges.
- Worked example: a 3×3 patch [10 10 200] in every row gives G_x = (200 + 400 + 200) – (10 + 20 + 10) = **760**; a uniform patch gives **0**.
- `CV_64F` keeps negative gradients (direction of change).
- First derivative peak = edge strength (Sobel, Canny). Second derivative **zero crossing** = edge location (LoG). LoG pipeline: Gaussian blur → Laplacian → zero-crossing detection.

8.3 Hough transform for lines

Every edge pixel votes for all lines that could pass through it; lines with many votes are real. The manual introduces $y = mx + b$ (parameter space m, b), but vertical lines have infinite slope, so OpenCV uses the normal form:

$$\rho = x \cos \theta + y \sin \theta, \quad \forall \theta \in [0, \pi]$$

Each pixel becomes a sinusoid in (ρ, θ) space; where many sinusoids cross is a line. Steps: edge detection $\rightarrow$ voting into an accumulator $\rightarrow$ peak detection $\rightarrow$ back-transform to image lines.

```
img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150, apertureSize=3)

# Standard Hough: infinite lines as (rho, theta)
lines = cv2.HoughLines(edges, 1, np.pi / 180, 150)      # rho res 1 px, theta
res 1 deg, min votes 150
std = img.copy()
if lines is not None:                                   # returns None when
nothing found!
    for rho, theta in lines[:, 0]:
        a, b = np.cos(theta), np.sin(theta); x0, y0 = a * rho, b * rho
        p1 = (int(x0 + 1000 * (-b)), int(y0 + 1000 * a)); p2 = (int(x0 - 1000
* (-b)), int(y0 - 1000 * a))
        cv2.line(std, p1, p2, (0, 0, 255), 2)

# Probabilistic Hough: finite segments (x1, y1, x2, y2), usually what you
want
segs = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=80, minLineLength=50,
maxLineGap=10)
prob = img.copy()
if segs is not None:
    for x1, y1, x2, y2 in segs[:, 0]:
        cv2.line(prob, (x1, y1), (x2, y2), (0, 255, 0), 2)
show([img, edges, std, prob], ["Original", "Canny", "HoughLines",
"HoughLinesP"])
```

Tuning: more lines $\rightarrow$ lower `threshold` or `minLineLength`; broken lines joined $\rightarrow$ raise `maxLineGap`. Keep only near-horizontal/vertical: `angle = np.degrees(np.arctan2(y2 - y1, x2 - x1))`, keep `abs(angle) < 10` or `abs(abs(angle) - 90) < 10`.

8.4 Hough transform for circles

$$(x - a)^2 + (y - b)^2 = r^2 \quad \text{\textit{3-D parameter space}} \quad (a, b, r)$$

```

gray = cv2.medianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 5)      # blur is
essential: fewer false circles
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=40,
                           param1=100, param2=30, minRadius=15, maxRadius=80)
# dp: accumulator resolution (1 = same as image), minDist: between centres,
# param1: Canny high threshold, param2: accumulator votes (LOWER = more
circles, more false ones)
out = img.copy()
if circles is not None:
    for x, y, r in np.round(circles[0]).astype(int):
        cv2.circle(out, (x, y), r, (0, 255, 0), 2); cv2.circle(out, (x, y),
2, (0, 0, 255), 3)
print("circles:", 0 if circles is None else circles.shape[1])

```

8.5 SIFT: the full matching pipeline

Stage	What happens
1 Scale-space extrema	Blur at increasing σ , subtract neighbours: $\text{DoG} = L(x, y, k\sigma) - L(x, y, \sigma)$. Search across scales so features are found whether large or small.
2 Keypoint localisation	Candidate must beat all 26 neighbours (8 same scale + 9 above + 9 below). Refine by fitting a 3-D quadratic; drop low-contrast and edge points.
3 Orientation assignment	Dominant gradient direction in the neighbourhood (36-bin histogram) $\rightarrow$ rotation invariance.
4 Descriptor	16×16 patch $\rightarrow 4 \times 4$ sub-regions $\times$ 8-bin orientation histograms = 128 numbers , robust to lighting.
5 Matching	Euclidean distance between descriptors (BFMatcher / FLANN), Lowe's ratio test.
6 Outlier rejection	RANSAC keeps only geometrically consistent matches.
7 Homography	Remaining inliers $\rightarrow H$ for recognition or stitching.

```

\text{DoG}(x,y,\sigma) = L(x,y,k\sigma) - L(x,y,\sigma), \quad \text{keep}
\text{match if } \frac{d_1}{d_2} < 0.75

```

Ratio test intuition: if the best match is barely better than the second best, the feature is ambiguous (repeated texture), so drop it.

```
def sift_match(img1, img2, ratio=0.75, min_matches=10, use_flann=False):
    """img1 = reference/object, img2 = scene. Returns H (img1 -> img2), good
    matches, inlier mask, keypoints."""
    sift = cv2.SIFT_create() # optional:
    SIFT_create(nfeatures=1000)
    g1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY) if img1.ndim == 3 else img1
    g2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY) if img2.ndim == 3 else img2
    kp1, des1 = sift.detectAndCompute(g1, None)
    kp2, des2 = sift.detectAndCompute(g2, None)
    if des1 is None or des2 is None or len(kp2) < 2: return None, [], None,
    kp1, kp2
    if use_flann:
        matcher = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5),
dict(checks=50)) # KD-tree
    else:
        matcher = cv2.BFMatcher(cv2.NORM_L2) # L2 for SIFT;
        NORM_HAMMING for ORB
    knn = matcher.knnMatch(des1, des2, k=2) # two nearest
    neighbours each
    good = [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio *
p[1].distance] # Lowe's test
    if len(good) < min_matches: return None, good, None, kp1, kp2
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0) # 5 px
    reprojection tolerance
    return H, good, mask, kp1, kp2

def draw_box(scene, ref, H, label=None, color=(0, 255, 0)):
    h, w = ref.shape[:2]
    corners = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h -
1]]).reshape(-1, 1, 2)
    box = cv2.perspectiveTransform(corners, H) # project
    reference corners into scene
    out = cv2.polylines(scene.copy(), [np.int32(box)], True, color, 3)
    if label: cv2.putText(out, label, tuple(np.int32(box[0, 0])),
cv2.FONT_HERSHEY_SIMPLEX, 0.9, color, 2)
    return out

# Keypoints only (manual 5.7)
```

```

image = cv2.imread("image.jpg")
kp, des = cv2.SIFT_create().detectAndCompute(cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY), None)
rich = cv2.drawKeypoints(image, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS) # circles = scale, line =
orientation
print("Number of keypoints:", len(kp), "Descriptor shape:", des.shape) #
(N, 128)

# Matches visualised (inliers only)
ref, scene = cv2.imread("object.jpg"), cv2.imread("scene.jpg")
H, good, mask, kp1, kp2 = sift_match(ref, scene)
if H is not None:
    vis = cv2.drawMatches(ref, kp1, draw_box(scene, ref, H), kp2, good, None,
                        matchesMask=mask.ravel().tolist(),
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    show([vis], [f"{int(mask.sum())} inliers / {len(good)} good matches"])
else:
    print("Object not found:", len(good), "good matches")

```

ORB is the free, faster alternative (binary descriptor): `cv2.ORB_create(1000)` with `cv2.BFMatcher(cv2.NORM_HAMMING)`. SURF is patented and missing from standard builds.

8.6 Task: computer screen detection (Hough lines) + document scanner

Screens are bright or dark rectangles bounded by long straight edges. Strategy: Canny → Hough segments for evidence of edges → contour polygons with 4 corners and screen-like aspect ratio → status by mean brightness.

```

def order_points(pts):
    """Return 4 points ordered TL, TR, BR, BL."""
    pts = pts.reshape(4, 2).astype(np.float32)
    s, d = pts.sum(axis=1), np.diff(pts, axis=1).ravel() # x+y and y-x
    return np.float32([pts[np.argmin(s)], pts[np.argmin(d)],
pts[np.argmax(s)], pts[np.argmax(d)]])

def four_point_warp(img, pts):
    rect = order_points(pts); t1, tr, br, bl = rect
    W = int(max(np.linalg.norm(br - bl), np.linalg.norm(tr - t1)))
    Hh = int(max(np.linalg.norm(tr - br), np.linalg.norm(t1 - bl)))
    M = cv2.getPerspectiveTransform(rect, np.float32([[0, 0], [W - 1, 0], [W
- 1, Hh - 1], [0, Hh - 1]]))
    return cv2.warpPerspective(img, M, (W, Hh))

```

```

img = cv2.imread("lab.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi / 180, 80, minLineLength=60,
maxLineGap=10)
line_mask = np.zeros_like(gray)
if segs is not None:
    for x1, y1, x2, y2 in segs[:, 0]:
        ang = abs(np.degrees(np.arctan2(y2 - y1, x2 - x1)))
        if ang < 10 or abs(ang - 90) < 10 or ang > 170:          # keep
horizontal + vertical only
            cv2.line(line_mask, (x1, y1), (x2, y2), 255, 3)
line_mask = cv2.morphologyEx(line_mask, cv2.MORPH_CLOSE, np.ones((7, 7),
np.uint8)) # join corners
cnts, _ = cv2.findContours(line_mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
out, screens = img.copy(), 0
for c in sorted(cnts, key=cv2.contourArea, reverse=True):
    area = cv2.contourArea(c)
    if area < 0.005 * gray.size: continue
    approx = cv2.approxPolyDP(c, 0.03 * cv2.arcLength(c, True), True)
    x, y, w, h = cv2.boundingRect(approx)
    if len(approx) == 4 and 1.1 < w / h < 2.2:                  # monitor-
like aspect ratio
        screens += 1
        inside = gray[y:y + h, x:x + w]
        status = "ON" if inside.mean() > 90 else "OFF"
        cv2.drawContours(out, [approx], -1, (0, 255, 0) if status == "ON"
else (0, 0, 255), 3)
        cv2.putText(out, f"S{screens} {status}", (x, y - 8),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
EXPECTED = 6                                                  # screens
in a full row
print(f"screens found: {screens}; missing: {max(EXPECTED - screens, 0)}")
show([img, edges, line_mask, out], ["Original", "Canny", "Hough H/V lines",
f"{screens} screens"])

```

Nested rectangles (bezel and display) can both pass; deduplicate by skipping boxes whose centre lies inside an already-accepted box. Document scanner: same contour step on `edges`, take the **largest** 4-point contour, then `four_point_warp(img, approx)`.

8.7 Task: asset tracking with SIFT (inventory)

Store one reference photo per asset; in each scene, the asset is present if enough RANSAC inliers match.

```
refs = {"Monitor-01": cv2.imread("data/monitor.jpg"), "Keyboard-01":
cv2.imread("data/keyboard.jpg"),
       "CPU-01": cv2.imread("data/cpu.jpg")}
scene = cv2.imread("data/lab_desk.jpg")
inventory, out = {}, scene.copy()
for name, ref in refs.items():
    H, good, mask, _, _ = sift_match(ref, scene, ratio=0.75, min_matches=12)
    inliers = 0 if mask is None else int(mask.sum())
    found = H is not None and inliers >= 12
    inventory[name] = {"present": found, "good": len(good), "inliers":
inliers}
    if found: out = draw_box(out, ref, H, label=name)
import pandas as pd
print(pd.DataFrame(inventory).T)
show([out], ["Detected assets"])
```

Why SIFT: identical-looking models are told apart by stickers, logos, asset tags (local texture), and detection survives distance (scale), camera angle (rotation) and partial occlusion.

8.8 Task: sensor anomaly detection with wavelets

Reproduces the expected output: top plot sensor data + denoised signal, bottom plot residuals with anomalies as green dots.

```
import pywt, numpy as np, matplotlib.pyplot as plt
np.random.seed(0)
N = 1000
sensor = np.random.normal(0, 1, N) # sample
sensor stream
sensor[[150, 420, 700, 880]] += [6, -5, 7, -6] #
injected faults (real anomalies)
# or: sensor = pd.read_csv("sensor.csv")["value"].to_numpy()

coeffs = pywt.wavedec(sensor, "db4", level=4)
sigma = np.median(np.abs(coeffs[-1])) / 0.6745
lam = sigma * np.sqrt(2 * np.log(N))
denoised = pywt.waverec([coeffs[0]] + [pywt.threshold(c, lam, "soft") for c
```

```

in coeffs[1:]], "db4")[:N]
residuals = sensor - denoised
k = 2.0 #
sensitivity: 2 std (plot style); 3 = stricter
anoms = np.where(np.abs(residuals) > k * residuals.std())[0]
print(f"{len(anoms)} anomalies at indices: {anoms}")

fig, ax = plt.subplots(2, 1, figsize=(12, 7))
ax[0].plot(sensor, label="Sensor Data"); ax[0].plot(denoised, "--",
label="Denoised Signal")
ax[0].set_title("Sensor Data and Denoised Signal"); ax[0].legend()
ax[1].plot(residuals, "r", label="Residuals"); ax[1].scatter(anoms,
residuals[anoms], c="g", zorder=3, label="Anomalies")
ax[1].set_title("Residuals and Detected Anomalies"); ax[1].legend()
plt.tight_layout(); plt.savefig("output/anomalies.png"); plt.show()

```

Real-time version: keep a sliding window (e.g. last 256 samples), recompute per new sample, flag the newest point if its residual exceeds $k \cdot \sigma$. Explain: denoising removes ordinary noise; whatever the clean model cannot explain (the residual) and is unusually large is abnormal.

8.9 Task: object recognition in video (SIFT)

```

ref = cv2.imread("data/object.jpg")
sift = cv2.SIFT_create()
kp_r, des_r = sift.detectAndCompute(cv2.cvtColor(ref, cv2.COLOR_BGR2GRAY),
None) # compute ONCE
bf = cv2.BFMatcher(cv2.NORM_L2)
h, w = ref.shape[:2]
corners = np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h -
1]]).reshape(-1, 1, 2)
cap = cv2.VideoCapture("data/test_video.mp4") # or a list of test
images
while True:
    ok, frame = cap.read()
    if not ok: break
    kp_f, des_f = sift.detectAndCompute(cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY), None)
    text, color = "NOT FOUND", (0, 0, 255)
    if des_f is not None and len(kp_f) >= 2:
        good = [m for m, n in (p for p in bf.knnMatch(des_r, des_f, k=2) if
len(p) == 2) if m.distance < 0.75 * n.distance]

```

```

        if len(good) >= 10:
            src = np.float32([kp_r[m.queryIdx].pt for m in good]).reshape(-1,
1, 2)
            dst = np.float32([kp_f[m.trainIdx].pt for m in good]).reshape(-1,
1, 2)
            H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
            if H is not None and mask.sum() >= 8:
                cv2.polylines(frame,
[ np.int32(cv2.perspectiveTransform(corners, H)) ], True, (0, 255, 0), 3)
                text, color = f"FOUND ({int(mask.sum())} inliers)", (0, 255,
0)
            cv2.putText(frame, text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
            cv2.imshow("Recognition", frame)
            if cv2.waitKey(1) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()

```

Speed-up: resize frames to 640 px width, or process every 2nd frame. Test-image version:
loop over `glob.glob("data/test_*.jpg")` and use `sift_match` + `draw_box`.

8.10 Task: panorama stitching

```

left, right = cv2.imread("data/left.jpg"), cv2.imread("data/right.jpg")    #
overlapping, taken while panning
H, good, mask, _, _ = sift_match(right, left)                               # maps RIGHT image
into LEFT image's frame
if H is None: raise SystemExit("not enough overlap/matches")
h1, w1 = left.shape[:2]; h2, w2 = right.shape[:2]
pano = cv2.warpPerspective(right, H, (w1 + w2, max(h1, h2)))    # wide canvas
pano[0:h1, 0:w1] = left                                         # paste the
reference image on top
# crop black borders
g = cv2.cvtColor(pano, cv2.COLOR_BGR2GRAY)
x, y, w, h = cv2.boundingRect(cv2.findNonZero(g))
pano = pano[y:y + h, x:x + w]
show([left, right, pano], ["Left", "Right", f"Panorama ({int(mask.sum())}
inliers)"], cols=3)
cv2.imwrite("output/panorama.png", pano)

# One-liner OpenCV stitcher (handles many images, blending, exposure):
status, pano2 = cv2.Stitcher_create(cv2.Stitcher_PANORAMA).stitch([left,
right])
print("Stitcher OK" if status == cv2.Stitcher_OK else f"Stitcher failed:
{status}")

```

More than two images: stitch left-to-right pairwise, each time treating the growing panorama as `left`. Needs roughly 30% overlap. Visible seam? Blend the overlap with `cv2.addWeighted` instead of pasting.

8.11 Task: lane detection with Hough lines

```

def lane_lines(frame):
    h, w = frame.shape[:2]
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
    roi = np.array([[(int(0.1 * w), h), (int(0.45 * w), int(0.6 * h)),
                     (int(0.55 * w), int(0.6 * h)), (int(0.95 * w), h)]],
np.int32)    # road trapezoid
    mask = np.zeros_like(edges); cv2.fillPoly(mask, roi, 255)
    masked = cv2.bitwise_and(edges, mask)
    segs = cv2.HoughLinesP(masked, 2, np.pi / 180, 50, minLineLength=40,
maxLineGap=100)

```

```

    left, right = [], []
    if segs is not None:
        for x1, y1, x2, y2 in segs[:, 0]:
            if x2 == x1: continue
            skip (infinite slope)
            m = (y2 - y1) / (x2 - x1); b = y1 - m * x1
            if abs(m) < 0.4: continue
            horizontal noise
            (left if m < 0 else right).append((m, b))
DOWN: left lane has negative slope
    overlay = np.zeros_like(frame)
    y_bot, y_top = h, int(0.6 * h)
    for group in (left, right):
        if group:
            m, b = np.mean(group, axis=0)
            one line per side
            x_bot, x_top = int((y_bot - b) / m), int((y_top - b) / m)
            cv2.line(overlay, (x_bot, y_bot), (x_top, y_top), (0, 0, 255),
10)
            return cv2.addWeighted(frame, 0.8, overlay, 1.0, 0), masked

road = cv2.imread("data/road.jpg")
result, masked = lane_lines(road)
show([road, masked, result], ["Road", "Canny inside ROI", "Detected lanes"])

```

For a video, call `lane_lines(frame)` inside the read loop (Section 2.8). Yellow lanes on grey roads: OR a yellow HSV mask with a white mask before Canny for robustness.

8.12 Task: coin detection and counting (Hough circles)

```

img = cv2.imread("data/coins.jpg")
gray = cv2.medianBlur(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 5)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=30,
                           param1=100, param2=30, minRadius=15, maxRadius=90)
# tested: 35 missed all 4 test coins, 30 found all 4
out = img.copy(); count = 0
if circles is not None:
    circles = np.round(circles[0]).astype(int)
    count = len(circles)
    radii = circles[:, 2]
    small = radii < np.median(radii)
classes

```

```

    for (x, y, r), s in zip(circles, small):
        cv2.circle(out, (x, y), r, (0, 255, 0) if s else (255, 0, 0), 3)
        cv2.putText(out, f"r{r}", (x - 15, y + 5), cv2.FONT_HERSHEY_SIMPLEX,
0.5, (0, 0, 255), 2)
cv2.putText(out, f"Coins: {count}", (10, 35), cv2.FONT_HERSHEY_SIMPLEX, 1.2,
(0, 0, 255), 3)
show([img, out], ["Original", f"{count} coins"])

# Cross-check with contours (works when coins do not touch)
_, bw = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
if np.mean(np.r_[bw[0], bw[-1]]) > 127: bw = cv2.bitwise_not(bw)
cnts, _ = cv2.findContours(bw, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("Hough:", count, "| contours > 300 px:", sum(cv2.contourArea(c) > 300
for c in cnts))

```

Missing coins → lower `param2` or widen the radius range. Extra circles → raise `param2`, raise `minDist` to about the smallest coin diameter. Touching coins defeat contour counting but not Hough (each circle votes separately); watershed (Section 7.8) is the region-based alternative. Value counting: map radius bands to denominations, `total = sum(values[band(r)])`.

8.13 Task: smart security zone (boundary detection in real-time video)

```

cap = cv2.VideoCapture(0)                                # webcam; or
"data/cctv.mp4"
bg = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=25,
detectShadows=True)
zone = np.array([[200, 150], [450, 150], [450, 400], [200, 400]], np.int32)
# restricted area (x, y)
while True:
    ok, frame = cap.read()
    if not ok: break
    fg = bg.apply(frame)                                # 255 = moving,
127 = shadow, 0 = background
    _, fg = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY) # drop shadows
    fg = cv2.morphologyEx(fg, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
    fg = cv2.dilate(fg, np.ones((5, 5), np.uint8), iterations=2)
    zone_mask = np.zeros(fg.shape, np.uint8); cv2.fillPoly(zone_mask, [zone],
255)
    cnts, _ = cv2.findContours(fg, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    alarm = False

```

```

    for c in cnts:
        if cv2.contourArea(c) < 800: continue          # ignore small
motion
        x, y, w, h = cv2.boundingRect(c)
        obj = np.zeros_like(fg); cv2.drawContours(obj, [c], -1, 255, -1)
        intrusion = cv2.countNonZero(cv2.bitwise_and(obj, zone_mask))  #
overlap with zone
        inside = intrusion > 0.2 * cv2.contourArea(c)
        alarm |= inside
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 0, 255) if inside
else (0, 255, 0), 2)
        cv2.polylines(frame, [zone], True, (0, 0, 255) if alarm else (255, 255,
0), 2)
        if alarm:
            cv2.putText(frame, "ALARM: UNAUTHORIZED OBJECT", (10, 40),
cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)
            print("\a ALARM", end="\r")                # terminal bell;
winsound.Beep(1000, 200) on Windows
            cv2.imshow("Security", frame); cv2.imshow("Foreground", fg)
            if cv2.waitKey(30) & 0xFF == ord("q"): break
cap.release(); cv2.destroyAllWindows()

```

Simpler alternative to MOG2: frame differencing against a reference frame, `diff = cv2.absdiff(gray, ref_gray)`, then threshold at about 25. Point-in-zone test for a centroid: `cv2.pointPolygonTest(zone, (cx, cy), False) >= 0` (1 inside, 0 on edge, -1 outside).

MOG2 needs a few dozen frames to learn the background; objects that stop moving slowly fade into it. For "objects placed and left", compare against a fixed empty-scene reference instead.

9. MCQ rapid-fire & function signatures

Q1 is MCQs: these are the facts most likely to be tested, then the exact signatures to copy under pressure.

9.1 Facts to know cold

Topic	Fact
Default colour order	<code>cv2.imread</code> returns BGR , uint8, shape (H, W, 3)

Topic	Fact
Bad path	<code>imread</code> returns <code>None</code> , no exception
<code>resize</code> <code>dsize</code>	(width, height); <code>.shape</code> is (height, width)
Gaussian kernel	Odd size; $\sigma = 0$ means computed from size
<code>threshold</code> return	(<code>retval</code> , <code>image</code>); with Otsu, <code>retval</code> is the chosen T
<code>equalizeHist</code> input	8-bit, single channel only
<code>cv2.add</code> vs <code>+</code>	Saturate at 255 vs wrap modulo 256
<code>addWeighted</code>	$\alpha \cdot \text{src1} + \beta \cdot \text{src2} + \gamma$, same size and type
Rotation angle sign	Positive = counter-clockwise on screen
<code>warpAffine</code> matrix	2×3 ; <code>warpPerspective</code> 3×3
Points needed	Affine 3 pairs, perspective 4 pairs
DOF	Translation 2, rigid 3, similarity 4, affine 6, projective 8
Affine preserves	Collinearity, parallelism, ratio of distances along a line
Projective breaks	Parallelism (lines converge to vanishing points)
Linear can't translate	$A \cdot 0 = 0$, origin is fixed
Homogeneous point	$(x, y) \rightarrow (x, y, 1)$; 2-D needs 3×3 , 3-D needs 4×4
Point vs geometric	Photometric changes values; geometric changes positions
Log transform	Expands dark values, compresses bright
Gamma < 1	Brightens; gamma > 1 darkens
Histogram equalization	Maps by CDF; flattens histogram; global
CLAHE	Local equalization with clip limit to stop noise blow-up
HSV hue range in OpenCV	0 to 179 (not 360)
Red in HSV	Two ranges: 0 to 10 and 170 to 179

Topic	Fact
HOG defaults	9 bins, 8×8 cells, 2×2 blocks, 0 to 180°, 3780-D for 64×128
LBP uniform bins	$P + 2$ (10 for $P = 8$)
SIFT descriptor	4×4×8 = 128-D; 26-neighbour extrema in DoG
Lowe's ratio	Keep if $d1/d2 < 0.75$ (0.7 to 0.8 common)
RANSAC	Robust model fit that rejects outlier matches
BFMatcher norm	<code>NORM_L2</code> for SIFT, <code>NORM_HAMMING</code> for ORB
Hough line space	(ρ, θ) , $\rho = x \cos \theta + y \sin \theta$
Hough circle space	3-D (a, b, r)
<code>HoughCircles</code> param2	Lower = more (false) circles
Canny stages	Blur → gradient → NMS → hysteresis
Canny thresholds	Above high = strong; between = weak, kept if connected; below low = dropped
LoG edges	Zero crossings of 2nd derivative
Sobel worked example	Gx on [10 10 200]×3 rows = 760
Otsu assumption	Bimodal histogram; maximises between-class variance
Adaptive blockSize	Odd, > 1
Watershed markers	-1 boundary, 1 background, 0 unknown before run
K-means input	float32, shape (N, features)
Opening / closing	Erode→dilate removes specks / dilate→erode fills holes
DWT output	Approximation (low) + detail (high) coefficients
CWT wavelet	Morlet for oscillations
Hardest DIP step	Segmentation
Enhancement vs restoration	Subjective look vs objective undo of degradation

9.2 Signatures

Function	Signature (key args)
imread	<code>cv2.imread(path, cv2.IMREAD_COLOR / IMREAD_GRAYSCALE / IMREAD_UNCHANGED)</code>
imwrite	<code>cv2.imwrite(path, img) -> bool</code>
cvtColor	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB / BGR2GRAY / BGR2HSV / GRAY2BGR)</code>
resize	<code>cv2.resize(img, (w, h))</code> or <code>cv2.resize(img, None, fx=, fy=, interpolation=)</code>
GaussianBlur	<code>cv2.GaussianBlur(img, (k, k), sigmaX)</code>
medianBlur	<code>cv2.medianBlur(img, k)</code> (odd int)
blur	<code>cv2.blur(img, (k, k))</code>
filter2D	<code>cv2.filter2D(img, ddepth=-1, kernel, borderType=)</code>
threshold	<code>ret, dst = cv2.threshold(gray, T, maxval, cv2.THRESH_BINARY [+ cv2.THRESH_OTSU])</code>
adaptiveThreshold	<code>cv2.adaptiveThreshold(gray, 255, ADAPTIVE_THRESH_MEAN_C / GAUSSIAN_C, THRESH_BINARY, blockSize, C)</code>
inRange	<code>cv2.inRange(hsv, lower, upper) → 0/255 mask</code>
equalizeHist	<code>cv2.equalizeHist(gray)</code>
CLAHE	<code>cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)</code>
calcHist	<code>cv2.calcHist([img], [ch], mask, [bins], [lo, hi])</code>
applyColorMap	<code>cv2.applyColorMap(gray8, cv2.COLORMAP_JET)</code>
LUT	<code>cv2.LUT(img, table256_uint8)</code>
add / addWeighted	<code>cv2.add(a, b)</code> , <code>cv2.addWeighted(a, alpha, b, beta, gamma)</code>

Function	Signature (key args)
bitwise	<code>cv2.bitwise_and(a, b, mask=m), bitwise_or, bitwise_xor, bitwise_not(a)</code>
getRotationMatrix2D	<code>cv2.getRotationMatrix2D((cx, cy), angle_deg, scale)</code>
warpAffine	<code>cv2.warpAffine(img, M2x3, (w, h), borderValue=)</code>
getAffineTransform	<code>cv2.getAffineTransform(src3x2_f32, dst3x2_f32)</code>
getPerspectiveTransform	<code>cv2.getPerspectiveTransform(src4x2_f32, dst4x2_f32)</code>
warpPerspective	<code>cv2.warpPerspective(img, H, (w, h))</code>
perspectiveTransform	<code>cv2.perspectiveTransform(pts_Nx1x2_f32, H)</code>
Sobel / Scharr	<code>cv2.Sobel(img, cv2.CV_64F, dx, dy, ksize=3), cv2.Scharr(img, cv2.CV_64F, dx, dy)</code>
Laplacian	<code>cv2.Laplacian(img, cv2.CV_64F, ksize=)</code>
Canny	<code>cv2.Canny(img, low, high, apertureSize=3, L2gradient=False)</code>
HoughLines	<code>cv2.HoughLines(edges, rho, theta, threshold) → (N, 1, 2) or None</code>
HoughLinesP	<code>cv2.HoughLinesP(edges, rho, theta, threshold, minLineLength=, maxLineGap=) → (N, 1, 4)</code>
HoughCircles	<code>cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp, minDist, param1=, param2=, minRadius=, maxRadius=) → (1, N, 3)</code>
findContours	<code>cnts, hier = cv2.findContours(bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)</code>
Contour measures	<code>contourArea, arcLength(c, True), boundingRect, minAreaRect, convexHull, moments, approxPolyDP(c, eps, True)</code>

Function	Signature (key args)
morphologyEx	<code>cv2.morphologyEx(m, cv2.MORPH_OPEN / CLOSE / GRADIENT, kernel, iterations=)</code>
distanceTransform	<code>cv2.distanceTransform(bin, cv2.DIST_L2, 5)</code>
connectedComponents	<code>n, labels = cv2.connectedComponents(bin)</code> (n includes background)
watershed	<code>markers = cv2.watershed(img_bgr, markers_int32)</code>
kmeans	<code>compact, labels, centers = cv2.kmeans(data_f32, K, None, criteria, attempts, cv2.KMEANS_PP_CENTERS)</code>
SIFT	<code>kp, des = cv2.SIFT_create().detectAndCompute(gray, None)</code>
Matchers	<code>cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d2, k=2), cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))</code>
findHomography	<code>H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)</code>
drawing	<code>cv2.line/rectangle(img, p1, p2, color, thick), cv2.circle(img, c, r, color, thick), cv2.putText(img, text, org, font, scale, color, thick), cv2.fillPoly(img, [pts], color), cv2.polylines(img, [pts], True, color, thick)</code>
video	<code>cv2.VideoCapture(path or 0), cap.read() -> (ok, frame), cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))</code>
skimage	<code>hog(img, orientations=9, pixels_per_cell=(8,8), cells_per_block=(2,2), visualize=True), local_binary_pattern(img, P, R, method="uniform")</code>

Function	Signature (key args)
pywt	<code>pywt.dwt(x, "db4"), wavedec(x, "db4", level=), waverec(c, "db4"), threshold(c, t, "soft"), cwt(x, scales, "mor1"), dwt2(img, "haar")</code>

9.3 Common errors and instant fixes

Error message (short)	Cause	Fix
<code>!_src.empty()</code> in <code>cvtColor</code>	Image failed to load (None)	Check path; if <code>img</code> is None
Sizes of input arguments do not match	Different sizes or channels in add/bitwise	<code>match()</code> from Section 2.2
<code>(-215) ... depth == CV_8U</code>	Float image passed to <code>equalizeHist/threshold/Canny</code>	<code>np.clip(x,0,255).astype(np.uint8)</code>
<code>TypeError: 'NoneType' ... not iterable/subscriptable</code>	Hough found nothing	<code>if lines is not None:</code>
cannot unpack ... expected 2	<code>findContours</code> return count (OpenCV 3 vs 4)	<code>cnts = cv2.findContours(...)[-2]</code>
<code>blockSize % 2 == 1 && blockSize > 1</code>	Even <code>blockSize</code> in <code>adaptiveThreshold</code>	Use odd values
<code>ksize.width > 0 && ksize.width % 2 == 1</code>	Even Gaussian kernel	Use odd values
Colours look blue-ish in <code>plt</code>	Forgot BGR → RGB	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>
Black or almost black image	Float range wrong, or gamma on 0 to 255 not 0 to 1	Normalise or clip properly
module 'cv2' has no attribute 'SIFT_create'	OpenCV older than 4.4	<code>pip install opencv-contrib-python</code> (before the exam)